

ModelFlow - Architecture and History

IB HANSEN

October 2, 2026

Contents

1	Introduction	1
1.1	What is in this book	1
1.2	How to read it	2
2	Historical Background: From MACRO and PCIM to ModelFlow	3
2.1	Overview	3
2.2	ADAM, TSP, MACRO and NASS	3
2.3	PCIM: the PC generation	5
2.4	A different problem	7
2.5	Continuity of the design idea	8
2.6	The same approach elsewhere	8
2.7	What changed: the scope of what can be delegated	9
2.8	A note on sources	10
3	ModelFlow: From Model Specification to Executable Code	11
3.1	Summary	11
3.2	An algebraic modeling language for root finding	11
3.3	Python is the user interface	13
3.4	The ModelFlow equation DSL	14
3.5	Higher-level model specification DSL	14
3.6	Preprocessors and models from other systems	15
3.7	Normalization	16
3.8	Tags and generated model transformations	18
3.9	Validate before rewriting	19
3.10	Regex and token rewriting	20
3.11	Outsourcing the language chain to Python	21
3.12	Runtime error localization	22
3.13	A worked example: specification to executable code	23
3.14	Layered view of the translation pipeline	24
4	ModelFlow: Model Structure and Solution	25
4.1	Summary	25
4.2	Model structure as a directed graph	25
4.3	Gauss-Seidel solution	27
4.4	Newton solution and the Jacobian	28
4.5	Stacked Newton solution for leaded endogenous variables	31
4.6	Solver choice	32
4.7	Worked example: from executable equation to graph and Jacobian	33
4.8	Solution architecture at a glance	34
5	ModelFlow: Local Stability and Model Dynamics	35
5.1	Summary	35
5.2	Linearization, the companion matrix and local stability	35

6	Stability in practice: a multiplier-accelerator model	39
6.1	The calculation in one paragraph	39
6.2	The model	39
6.3	Data	40
6.4	A function which simulates the model and calculates the eigenvalues	41
6.5	No amplification	42
6.6	Explosion	42
6.7	Exploding oscillations	43
6.8	Perpetual oscillations	44
6.9	Dampened oscillations	44
6.10	Jackknife experiments to identify the impact from each equation	45

Chapter 1

Introduction

This is a draft

This supplement is a **work in progress**. It is circulated to be read and argued with, not as a finished account.

Comments, corrections and suggestions are very welcome — on the substance, on the history, or on anything that is unclear, wrong or missing.

This supplement is about **how ModelFlow is put together** — what happens between an economic equation written by a modeller and a number produced by a solver, and why the chain is arranged the way it is.

The guiding principle is a division of labour:

Use Python syntax wherever possible, transform only the notation specific to economic modelling, and delegate the general language and numerical machinery to Python and its scientific libraries.

ModelFlow therefore does not implement an expression language, a compiler, a sparse linear algebra package or a graph library. It implements the part that is specific to economic models — lags and leads, endogenous and exogenous variables, add factors, normalization, dimensional expansion — and generates ordinary Python for everything else.

1.1 What is in this book

The four chapters follow one chain, and are ordered so that each takes over where the last leaves off.

Historical background comes first because it explains why the rest looks as it does. The same architecture was used on a UNIVAC 1100 mainframe around 1983 to move the ADAM model from TSP into Danmarks Nationalbank's NASS, and again in PCIM from 1985, which put ADAM on a PC with 640 KB of memory and an 8087 coprocessor. ModelFlow, begun around 2013 for an unrelated problem — unmaintainable bank stress-test models in Excel — re-derived it a third time. The chapter also notes that the approach is not unique to ModelFlow: Dynare and Project LINK arrived at much the same arrangement independently.

Specification to executable code takes the translation side. The higher-level DSL of lists, `do`, `doable` and `sum`; the preprocessors that bring in models from other systems; normalization into $y = F(y, x)$ form together with the add-factor, exogenizable and fitted variants; syntax validation against the equation the modeller actually wrote; the token rewriting that turns variable names into array positions; and compilation with `exec` or Numba.

Structure and solution starts from the executable equations and asks how to order and solve them. The dependency graph, its decomposition into prolog, simultaneous core and epilog, and the feedback vertex set that shrinks the problem that remains — then the solvers that use all of it: Gauss-Seidel, Newton, Newton reduced to the feedback variables, and stacked Newton when leads couple the periods together.

Local stability and dynamics turns the derivative machinery to a different purpose. The Jacobian built for a Newton step is a linearization, and it can be analysed instead of iterated on: contemporaneous effects solved out, lags assembled into a companion matrix, and its eigenvalues used to ask whether small disturbances decay, oscillate or grow.

1.2 How to read it

The history stands on its own and can be skipped by anyone who only wants to know how the system works today. The three technical chapters are meant to be read in order — the second begins from the executable equations the first produces, and the third from the Jacobian the second builds — but each states its own starting point, so they can also be consulted separately.

Two conventions are worth flagging. Generated code is treated throughout as an implementation detail that the modeller is not expected to read; where it appears, it is to show what the translation produces, not because anyone works at that level. And claims about what the software does are meant to describe what it actually does — where something is an intention rather than a tested capability, it is marked as such.

Readers interested in the specification language itself — the DSL, the `%%Makemymodel` magic, normalization and add factors in practical detail — will find them in the companion supplement on model specification.

Chapter 2

Historical Background: From MACRO and PCIM to ModelFlow

2.1 Overview

ModelFlow’s translation architecture has predecessors in earlier economic-model systems. A recurring design idea has been to transform only the notation specific to economic modelling and delegate the general-purpose language processing and numerical execution to an established programming language and compiler.

This history is kept separate from [From model specification to executable code](#), which describes the same idea in its present form.

The point of recording it is not nostalgia. The architecture now used by ModelFlow was not arrived at by designing for Python. It was first used around 1983, thirty years before ModelFlow was begun, and it has since been implemented three times in four different languages.

It is not, however, a story of continuous evolution, and the line is not even unbroken. PCIM, the second generation, was replaced in 2009 by Gekko, a C# system that took over the handling of ADAM. ModelFlow is not PCIM’s successor in that sense; that role belongs to Gekko. ModelFlow arose separately, four years later, from an unrelated practical problem — the maintainability of bank stress-test models built in Excel — and the older architecture was adopted because it turned out to answer that problem too. The same author wrote PCIM and ModelFlow, which is why the resemblance is not a coincidence; but the later system was a fresh start rather than a port, and it reached its design by re-deriving it rather than by inheriting it.

Nor was the architecture designed in the abstract at any point. Each generation was a response to an immediate practical difficulty:

Generation	The problem that forced it
MACRO / NASS , ~1983	A new version of ADAM that TSP could not handle, and TSP could not be modified
PCIM , 1985	ADAM was expensive to run and reachable only by the few institutions that could afford it
ModelFlow , ~2013	Bank stress-test models in Excel that had become un-maintainable

That an architecture first reached for under deadline on a mainframe should still be the right answer to spreadsheet maintainability in a bank thirty years later is reasonable evidence that it is a property of the *problem* — translating economic notation into something a machine can evaluate — rather than an artefact of any one language, machine or application.

2.2 ADAM, TSP, MACRO and NASS

Around 1983, at **RECKU**, the computer centre of the University of Copenhagen, the present author used UNIVAC MACRO to **connect the ADAM model from TSP to NASS**, the simulation

system at **Danmarks Nationalbank** developed by **Jørgen Petersen**.

2.2.1 The forcing problem

This was not an architectural exercise. ADAM had until then been handled in TSP. A new version of the model was then produced which **TSP could not handle**, and TSP was not a system that could be modified to accommodate it.

That left an immediate and practical difficulty: an existing model, a new version of it that the existing tool would not run, and no possibility of changing the tool. Moving the model to NASS was the way out, and it had to work.

The architecture described here was a response to that, arrived at under time pressure. It is worth keeping in mind when reading what follows, because the same is true of each later generation.

2.2.2 The mainframe setting

The work was done on a **UNIVAC 1100**, a mainframe on which simulation ran in **batch mode**. A run was submitted, waited for, and paid for: both the **cost** and the **elapsed time** of a simulation were significant.

This shapes everything about the period. Iteration was expensive, so a model run was something to be got right rather than repeated casually, and any error that surfaced only at run time was costly in a way that is difficult to appreciate now. It also makes the attraction of the following generation obvious: a PC, however limited, gave the modeller a machine that could be run again immediately and at no marginal cost.

2.2.3 What MACRO was

UNIVAC MACRO was a **pattern-matching and text-substitution language** available on the 1100 series, intended for **source-to-source transformation** — producing program source from other program source. RECKU published its own manual for it [Frentz, 1981], under a title that says plainly what it was taken to be: *Tekstbehandlingssystemet MACRO* — the text-processing system MACRO.

It is worth being clear about how little it knew. MACRO was not a modelling tool, had no notion of an equation, a variable or a lag, and no arithmetic interest in what it processed. It read text, recognized patterns in it, and emitted other text. Everything specific to economics had to be expressed as patterns to match and text to substitute in their place.

That turns out to be exactly the right amount of knowledge for the job, and the reason it was the right tool is the reason the same shape of tool is still used today: the task is not to understand the equations, only to re-express the parts of them that the target language does not already understand.

2.2.4 Translation into FORTRAN

NASS was written in FORTRAN. Economic equations therefore had to be transformed into a representation suitable for the NASS/FORTRAN environment: the model specification originated as ADAM in and from TSP, MACRO performed the pattern matching and substitution, FORTRAN was the target language, and NASS provided the simulation machinery.

The economic equations already contained most of the required expression structure — arithmetic, parentheses, precedence, function calls. None of that had to be re-implemented, because the FORTRAN compiler already knew how to handle it. What did *not* exist in FORTRAN was the model-specific notation: the meaning of a lag, the distinction between an endogenous and an exogenous variable, the mapping from a variable name to a location in the data. MACRO could concentrate on exactly that, and hand everything else to the compiler.

This is the division of labour that the current system still uses, with **ast** and the Python compiler in place of the FORTRAN compiler.

2.3 PCIM: the PC generation

PCIM — *PC Integrated Modelling* — moved this type of modelling architecture onto the PC. Written by the present author, it was started in **1985** and handled ADAM until **2009**, when **Gekko**, a system written in C#, took over.

Twenty-four years is a long life for a program written for an 8086, and the length of service is easier to show than to assert: the model group at Statistics Denmark was still publishing guides to running ADAM in it in the 1990s [Kristensen, 1996, Hansen, 1999], a decade after it was written.

2.3.1 Who could afford to run a model

The purpose was not primarily technical. It was to make using ADAM **cheap**, and thereby to make it **available to a far wider group of economists**.

On the mainframe, the cost and turnaround described above were not merely an inconvenience; they were a barrier to entry. Running the national model was in practice reserved for the few government bodies with the resources to do it. Everyone else could read about ADAM, and argue about its results, but could not put a question to it themselves.

A model that only a handful of institutions can run is a different kind of object from one that any competent economist can run — as an instrument of analysis, and as a subject of debate. Moving ADAM to the PC changed which of the two it was. Against the mainframe the PC gave up memory, speed and storage; what it offered in return was a model on the desk of anyone who wanted one, at no cost per run.

That trade is what the memory problem described next was the price of.

2.3.2 The machine, and what it could not do

PCIM was developed in 1985 on an **Olivetti M21**, fitted with an Intel **8086** and an **8087** floating-point coprocessor.

Both chips mattered. Early PCs did floating-point arithmetic in software, which for a model of ADAM's size would have been hopeless. The 8087 did it in hardware. The 8086, for its part, had a full 16-bit path to memory where the IBM PC's 8088 had only eight bits externally, so data access was correspondingly faster.

It is worth pausing on what that hardware could actually do. Depending strongly on the operation, an 8087 delivered floating-point performance on the order of **tens of thousands of operations per second**, enormously below a present-day processor. A national macroeconomic model was nevertheless solved, repeatedly and usefully, on that hardware.

Memory was the harder limit. The machines of the day offered **640 KB of RAM**, and neither ADAM's data nor the generated code came close to fitting in it. Underneath that sat the **64 KB segment** that 8086 addressing imposed, constraining how data could be addressed even within the memory that did exist.

That the whole thing was possible at all was not obvious at the time. The project began as a wager — **a box of beer that it could be done**. How it was won is the subject of *The solver* below, and the two decisions that won it outlived the constraint that forced them.

2.3.3 Two languages

PCIM itself was written in **FORTRAN**, while the **text-processing and substitution component was written in Pascal**. The numerical model and simulation machinery belonged naturally in FORTRAN; Pascal provided the textual front end that read the economic specification and produced FORTRAN from it.

The choice of two languages is the architecture showing through. The transformation stage and the execution stage have genuinely different requirements — string handling and pattern matching on one side, numerical throughput on the other — and each could be written in whatever language suited it. The boundary between them is generated source text, which any language can write and any compiler can read.

What that freedom was actually spent on here was the compiler. The FORTRAN was compiled with **Ryan-McFarland FORTRAN**, chosen for one specific reason: RM/FORTRAN supported **arrays larger than the 64 KB segment**. A model databank does not fit in 64 KB, so a compiler that could not exceed it was of no use whatever else it did well. Because the boundary was generated text, the front end neither knew nor cared which FORTRAN compiler consumed its output, and the compiler could be picked purely on what the machine demanded.

2.3.4 What the user saw

The two parts are visible in the contemporary user documentation, which describes PCIM as consisting of one — “which few users ever become acquainted with” — that turns a model into an executable file, and the executable itself, which understands the commands a user issues for model runs [Hansen, 1999]. A model was a `.frm` formula file compiled into a matching `.exe`; the May 1998 version of ADAM was `maj98.frm` and `maj98.exe`. That the translation stage was invisible to most users is exactly the intention: the generated code is an implementation detail.

The same handbook shows how much of the current feature set the system had acquired. Alongside simulation, PCIM offered **exogenization and endogenization**, **calculation of add factors**, and target-instrument analysis. These are the operational transformations described in [From model specification to executable code](#), under recognizably the same names, documented in 1999 — a decade and a half before ModelFlow re-derived them.

What the user saw changed more than what ran underneath it. The early system relied on conventional **FORTRAN READ and WRITE** interaction; later the front end moved to **Windows**, giving users a more modern way to prepare runs, inspect results and operate the model. The equation generator and the solver were left largely unchanged — the interface could be replaced without redesigning them, which is the same separation at work one level up.

2.3.5 The solver, and how 640 KB was won

The basic numerical method was conventional **Gauss-Seidel**, already old long before PCIM. The distinctive engineering problem was not inventing an iteration scheme but arranging a large econometric model so that repeated equation evaluation was fast and feasible on the available hardware.

The equations were generated as FORTRAN assignments and evaluated sequentially. As soon as an endogenous variable had been recalculated, its new value was available to equations later in the same sweep. That algorithm stayed in use for PCIM’s whole lifetime, from 1985 to 2009, while the hardware, the data precision and the user interface all changed around it.

Getting it into 640 KB took two decisions.

The first was to recognize that solving a period does not require the whole databank. It requires the current value of each variable and those lagged values that actually occur in the equations — and nothing else. PCIM therefore assembled a **solution vector** holding exactly that working set, and kept only it in memory while solving. The rest of the data stayed on disk.

The second was **segmentation of both code and data**, so that only the part of the generated code being executed, and only the data it needed, had to be resident at any moment.

Together they were enough. The wager was won — and won more comprehensively than the terms required, since PCIM went on to handle ADAM for the next twenty-four years.

Neither decision is a modelling idea; both are consequences of the machine. What became of them afterwards is more interesting than either.

2.3.6 What became of the two memory decisions

Segmentation went when the constraint went, and left nothing behind. The operating system does that work now, and the machinery for it is simply gone.

The solution vector did not go. As memory stopped being scarce the pressure that produced it went away, and something else was bought with the same money: under the original constraint the arithmetic was done in **single precision**, and once memory was no longer the binding limit it moved to **double**. That did more than add significant digits. The greater precision made the iterative solution

converge more cleanly and could cut the number of iterations enough to improve overall solving speed, despite each value taking twice the memory, and it allowed convergence tolerances to be set with less concern for rounding noise.

The vector survived anyway, for a reason that had nothing to do with the one that created it. Packing the values the equations need into one compact, contiguous block means consecutive equations read memory that is close together. On machines with a cache hierarchy that **improved the cache hit ratio** — the working set now fits in cache for much the same reason it once had to fit in RAM. A design forced by a 640 KB limit turned out to be a good design on machines with a thousand times more memory, because the relevant scarcity had moved up a level rather than disappeared.

It is still in the code. ModelFlow contains one Gauss-Seidel solver built on a stuffed one-period vector, computing a fixed offset for each variable and lag, gathering the working set into one contiguous array before iterating and writing only the endogenous results back afterwards. Its stated advantages are the 1985 ones: compile-time offsets and data locality.

It is an experiment rather than the working default, though. The ordinary solvers index the full two-dimensional array directly, and on modern machines that is fast enough that the extra gather-and-scatter has not earned its place in the mainline.

So of the two decisions that won the beer, one is gone and the other is kept as a curiosity. What actually survived into everyday use is neither of them, but the habit of thought behind them — the subject of *Continuity of the design idea* below.

2.4 A different problem

The lineage described so far ends in 2009 with Gekko — a normal succession, and not the origin of what follows.

ModelFlow began some four years later, around **2013**, and somewhere else entirely: from the **maintainability of bank stress-test models built in Excel**.

A spreadsheet is an excellent calculator and a poor model specification. The economic logic is distributed across cells rather than written down anywhere; the specification, the data and the presentation occupy the same object; and repeated structure — the same relationship across many portfolios, exposures or scenarios — has to be produced by copying formulas, after which the copies are independent and can silently diverge. None of this matters much for a small calculation. For a stress-test model that has to be reviewed, audited, re-run under many scenarios and handed to someone else, it matters a great deal.

Stated that way, the problem is recognizably the one the older architecture had always addressed: separate the economic specification from its implementation, write each relationship once, and generate whatever the machine needs from that single authoritative statement.

2.4.1 Why not restart PCIM

The obvious move would have been to revive the existing system, which already embodied the answer — by then no longer in service, and so in principle available.

It was rejected for the same reason that had motivated the exercise in the first place. Bringing a Pascal and FORTRAN system from the 1980s back into service would have created a maintainability problem of its own — and solving a maintainability problem by taking on another is not progress.

The alternative had also become considerably more attractive in the intervening years. Much of what PCIM had been obliged to build for itself was by then available off the shelf in Python — the extent of it is set out under *What changed* below — while the part that genuinely had to be written, the model-specific translation, was the part the architecture had always isolated anyway.

So the architecture was carried forward and the implementation was not. ModelFlow is a new system rather than a translation of the old one — what it inherits is a design, not a code base.

2.5 Continuity of the design idea

The implementation technologies changed completely, and the third entry is not a continuation of the second — ModelFlow was written from scratch, for a different purpose. The architecture nonetheless remained recognizable.

Generation	Economic specification	specifications	Transformation	Target language	Runtime
UNIVAC / NASS , from ~1983	ADAM from TSP	equations	MACRO	FORTTRAN	NASS
PC / PCIM , 1985 – 2009	Economic equations		Pascal text processing	FORTTRAN	PCIM
Python / ModelFlow , from ~2013	ModelFlow imported model	DSL or model	Expansion, normalization, AST validation, token rewriting	Python (optionally Numba)	ModelFlow

The first two generations targeted macroeconomic models — ADAM in both cases. The third began from bank stress testing and has since been applied to macroeconomic models as well, which is the reverse of the path one might expect and a further sign that the architecture is not tied to a particular class of model.

The recurring principle is:

Transform the economic notation that is specific to modelling, and delegate the general-purpose programming-language work to an existing compiler or language implementation.

Four things stayed constant across all three generations:

- the **boundary is generated source text**, not a shared in-memory data structure, which is what allows the two sides to be written in different languages and replaced independently;
- the transformation stage handles **only** the model-specific notation, never the general expression grammar;
- the modeller writes economic equations, and the generated code is treated as an implementation detail they are not expected to read;
- **variable names are resolved to fixed positions once**, while the code is being generated, so that solving is arithmetic on numbered slots rather than repeated lookup by name.

The last of these is worth drawing out, because it is easy to mistake for an implementation detail of whichever language is in use. It is not. The FORTRAN arrays of the earlier systems and the NumPy array the current system indexes as `values[row, 42]` embody the same decision: a name is something the *modeller* uses, a position is something the *machine* uses, and the translation between them belongs at code-generation time — once — rather than in the inner loop.

Note that this is the durable inheritance, and it is a more modest claim than it might first appear. What carried forward is resolving names to positions, which every ModelFlow solver does. The particular way PCIM arranged those positions — one contiguous working-set vector — did not carry forward into normal use, and did not need to.

2.6 The same approach elsewhere

Nothing described here is a private invention, and it would be misleading to present it as one. Translating an economic model into a general-purpose programming language, and letting that language’s compiler do the rest, is a recognized way of building modelling systems, arrived at independently by others.

Dynare is the clearest contemporary example. A model is written in Dynare’s own `.mod` notation; a preprocessor reads it and **generates MATLAB/Octave code**, which is then executed [Adjemian et al., 2011]. The economic notation is Dynare’s; the expression language, the numerical libraries and

the runtime are MATLAB's. That is precisely the division of labour described in this document, with a different source notation and a different target language.

The resemblance goes further than the split itself. The generated files include a **driver** that runs the computing tasks, and separate static and dynamic model files that return the residuals of the equations **and their Jacobian** — the same derivative-generation step described in [Model structure and solution](#), arrived at for the same reason. And the preprocessor takes an option `language=matlab|julia`, which is the clearest possible demonstration of what the architecture buys: when the boundary is generated source text, the target language becomes a setting rather than a rewrite.

Project LINK, the international linked-model project, belongs to the same tradition. Petersen and Sims [1987] describe its solution machinery, and the resemblance to what is described in this document extends well past the translation step.

Two points from that account are worth singling out.

The first is the **solver hierarchy**: Jacobi, Gauss-Seidel and Newton, applied according to how difficult the system proves to be. That is, in outline, the arrangement ModelFlow arrived at independently — Gauss-Seidel as the ordinary method for models that suit it, Newton reserved for the harder nonlinear and simultaneous cases. The reasoning behind the hierarchy is the same in both: Newton is more powerful and more expensive, so it is worth using only where cheaper iteration fails.

The second is that LINK was solved on an **IBM 3090** with a **parallel and vector implementation**. The specific techniques have dated, but the underlying position has not: a model system that generates its own computational code is in a good position to exploit whatever the hardware of the day rewards, because the generator can emit whatever form the machine prefers. What was vector code on a 3090 is machine code from a just-in-time compiler now, and the reason both are available is the same — the equations are text at the point where the decision is made.

None of this is folklore, either. The numerical side of it — the iterative methods, the ordering and decomposition of large systems, the sparse techniques — is documented as a subject in its own right, most fully by Pauletti [1997], whose account of solving large macroeconomic models covers much the same ground as [Model structure and solution](#).

The convergence is the interesting part. These systems were built by different people, in different countries, for different models, in different decades, with no particular reason to imitate one another. That they arrived at the same arrangement — the same separation of specification from execution, and in LINK's case the same hierarchy of solvers on top of it — is better evidence for the argument made in this document than the persistence of a single lineage could ever be. The architecture is a response to the shape of the problem, and people who take the problem seriously tend to end up somewhere near it.

There is a neat consequence. ModelFlow can **import and solve Dynare models**, so far as they fall within what it can handle.

That it works at all follows from the shared architecture. Both systems keep the specification separate from the machinery that runs it. Each leaves a specification the other can read.

2.7 What changed: the scope of what can be delegated

The principle survived unchanged, but its leverage grew, because the amount of work that can be handed to someone else grew. This was the deciding argument against restarting PCIM, and it is worth setting out in full.

In the FORTRAN generations, essentially one thing was delegated: parsing and compiling the expression language. Everything else had to be written, and — as the 640 KB problem shows — that included work at a considerable distance from economics. Deciding which values constitute the working set, segmenting code and data, and orchestrating what was resident at any moment were all part of building a model system, because nothing else was going to do them.

The current system delegates considerably more, and to specialized libraries rather than to a single compiler:

Task	Delegated to
Expression grammar, compilation, execution	Python
Machine-code compilation of the generated equations	Numba
Symbolic differentiation and equation rearrangement	SymPy
Graph decomposition, ordering, feedback sets	NetworkX
Sparse matrix storage and LU factorization	SciPy
Data management, periods, series alignment	pandas
Visualization and reporting	the wider Python ecosystem
Memory residency and paging	the operating system

This is why the current system can offer things that had no counterpart in the earlier generations — automatic Jacobians from symbolic differentiation, prolog/core/epilog decomposition and feedback-set reduction, sparse and stacked Newton solution for forward-looking models — without a corresponding growth in the amount of model-specific code that has to be maintained.

The architecture did not change to make this possible. It simply became possible to apply the same architecture to more of the problem.

The last row of that table is the one with a moral. Both of PCIM’s memory decisions were answers to a constraint the operating system now handles, and both have accordingly lapsed from everyday use, as described above. Neither was kept out of loyalty to a design that once mattered.

That is the harder half of maintaining a long-lived architecture. The principle at its centre was worth carrying across four decades and three implementations; a good deal of what was built around it was worth letting go — including, in the end, the implementation itself.

2.8 A note on sources

MACRO is documented in Frenzt [1981], Project LINK in Petersen and Sims [1987], and the numerical methods generally in Pauletto [1997]. PCIM has surviving user documentation from the model group at Statistics Denmark: the handbook [Hansen, 1999], written by the present author but published without attribution, and Tony M. Kristensen’s introduction to running ADAM in it [Kristensen, 1996]. Statistics Denmark has confirmed that these may be cited here.

NASS is not documented, and none of these systems was written up as a description of its design. What is said about their architecture rests on the recollection of the person who built them; dates and details should be read in that light.

The current architecture continues in [From model specification to executable code](#), [Model structure and solution](#), and [Local stability and model dynamics](#).

Chapter 3

ModelFlow: From Model Specification to Executable Code

3.1 Summary

ModelFlow uses a deliberately lightweight and layered architecture for specifying, expanding, normalizing and translating economic models into executable code.

The central principle is:

Use Python syntax wherever possible, transform only the notation specific to economic modelling, and delegate the general language machinery to Python and its scientific ecosystem.

This chapter follows the translation side of the architecture: the ModelFlow DSL and imported model formats, dimensional expansion, normalization, tags and generated variants, AST validation, token rewriting, and execution as Python or Numba-compiled code. It also sets out what the DSL deliberately leaves out, because the user interface around the equations is Python itself. The next chapter, [Model structure and solution](#), starts from the executable equation system and explains graph ordering, Gauss-Seidel and Newton solution. [Local stability and model dynamics](#) then uses the derivative machinery for stability analysis.

The historical development that preceded ModelFlow—MACRO/NASS and PCIM—is described separately in [Historical background](#).

3.2 An algebraic modeling language for root finding

For a reader who knows GAMS, AMPL or Pyomo, the shortest description of what follows is that ModelFlow is an **algebraic modeling language**, pointed at a different problem class.

The family traits are all here: declarative algebraic notation for the equations; sets and indexed expansion, so that one written relationship becomes many concrete ones; specification kept separate from the data and from the solver; derivatives generated by the system rather than supplied by the modeller; and several solvers selectable against the same unchanged specification.

What differs is what is being solved. An algebraic modeling language is normally aimed at optimization — an objective subject to constraints. ModelFlow is aimed at a **square system with no objective**: as many equations as endogenous variables, solved for the values that make them hold simultaneously. In GAMS’s own vocabulary that is a **CNS**, a constrained nonlinear system, rather than an LP or an NLP. Root finding is therefore not an implementation detail; it is the model class the language exists to express. Two later sections read more easily with that in mind: *Normalization* rearranges each equation so that one variable stands alone on the left, which is the form fixed-point

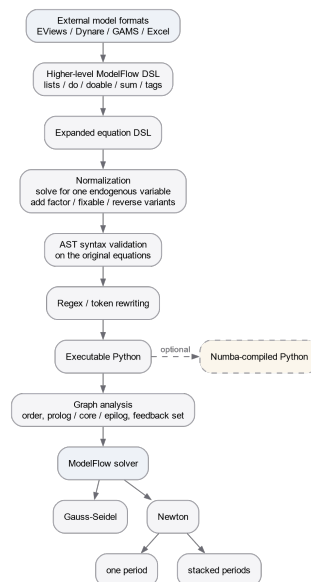


Figure 3.1: The complete chain. This chapter covers specification through executable code; model ordering and solution are treated in the next chapter.

iteration can use directly, and *Exogenization* swaps which variables are unknown and which are given — in a square system, simply a different choice of what to solve for.

Two things set ModelFlow apart from the mainstream of that family. **Time is first-class** rather than modelled as a set: lags and leads are notation, and the solution runs recursively over periods, stacked only where leads require it — which is what the ordering and decomposition in [Model structure and solution](#) exist to exploit. And ModelFlow **generates and executes its own code** rather than emitting an instance for a third-party solver, delegating the linear algebra to SciPy but not the nonlinear solution. In that second respect it resembles Dynare, which likewise generates code; the comparison is drawn in [Historical background](#).

3.2.1 Why the language can be smaller

The narrower target is what makes the smaller language possible.

A general algebraic modeling language has to serve linear programs, nonlinear programs, complementarity problems and mixed-integer models, in any application domain, and its notation has to be general enough to carry all of that. ModelFlow aims at one class: **time series models**, in which the data is indexed by period and the equations say how the variables move from one period to the next.

Econometric models are the origin of that class and still the main case — macroeconomic models, stress-test models, projection models — but the class is wider than econometrics and ModelFlow is not restricted to it. Anything that is a system of difference equations in discrete time has the same shape and is handled the same way: epidemiological contagion models, demographic and cohort projections, general time-oriented difference systems, and differential systems written in discretized form. What these share is not their subject matter. It is that a period’s state is a function of earlier periods, which is exactly what the notation and the solvers are built around.

Narrowing the target means the specification can be *simplified* rather than generalized. Sets exist because models are disaggregated by sector, age or country, not because a general index mechanism is wanted for its own sake. Time needs no set at all, because there is only ever one time dimension. The declarations that a general language requires — variable types, domains, model membership — mostly follow from the equations themselves and can be inferred. What is left on the page is close to the algebra an economist would write anyway, which is the whole point: a specification that stays parsimonious and stays readable as economics.

Drud [1983] drew the same boundary, for the same reason. Surveying sixteen modelling systems, he selected them on exactly this criterion — only systems based on time series models — because that

is what makes such systems comparable at all. The ones he compared are the ancestors of the ones ModelFlow is compared with here: TSP, TROLL, MODELEASY, SAS/ETS, XSIM.

His paper is also an early and unusually clear statement of a trade-off that recurs throughout this chapter. Drud classifies the equation formats a modelling system will accept as `var = expr`, `f(var) = expr` with invertible `f`, `expr = 0`, and the fully general `expr1 = expr2`. The choice is not independent of the solution algorithm. Gauss-Seidel needs a variable standing alone on the left, so it needs one of the first two forms; Newton and minimization methods take the general ones. A system that accepts general equations and wants to use Gauss-Seidel must therefore normalize them, and the normalization is not part of the model — it is information the algorithm needs and the model does not carry. That is exactly what the *Normalization* section below does, four decades later, and the reason it is a separate stage rather than something the modeller is asked to supply.

3.3 Python is the user interface

The DSL described in this chapter covers one thing: the equations. Everything around them — reading data, constructing scenarios, driving runs, inspecting results, producing charts and tables — is done in **Python**, against the model object, in a language ModelFlow did not have to design.

This is the second half of the narrowing argument above.

A self-contained modelling system has to grow its own everything. The systems in Drud’s survey each acquired a command language, then a data-management layer, then loops and conditionals, then a table and reporting facility, then graphics, then some way of calling out to code the system could not express. None of that is modelling. It is general-purpose programming, arrived at one requirement at a time, and every system’s version of it has to be separately designed, documented, maintained and learned.

ModelFlow does none of it. The specification language stops at the equation, and the work surrounding it is ordinary Python:

- **Data wrangling** is pandas. A model’s data is a DataFrame, so joins, resampling, interpolation, and reading from Excel, SQL, Parquet or a web API are all available without ModelFlow implementing any of them.
- **Control structures** are Python’s. A scenario sweep is a `for` loop, a conditional run is an `if`, a family of variants is a list comprehension. There is no ModelFlow macro language, because there is nothing left for one to do.
- **Updating and scenario construction** are assignment and arithmetic on the data, written in the same notation as the rest of the analysis.
- **Reporting and visualization** are matplotlib, plotly and the notebook. ModelFlow adds model-aware convenience on top — a variable charted across scenarios, an attribution table — but these are thin layers over the standard libraries, not a report-writing language of their own.

The gain is not only access to the ecosystem, considerable though that is: statistical libraries, optimizers, machine-learning tools, testing frameworks, version control, and a language most quantitative analysts already know. The gain is equally in **what never had to be built**. Each of those facilities is a subsystem ModelFlow did not design, does not document and cannot break, which is a large part of why the framework is as small as it is.

The division is clean. ModelFlow supplies what is specific to models: equations, expansion, normalization, ordering, solvers, attribution. Python supplies what is general. That is the same boundary that runs through the rest of this chapter, applied to the user’s side of the system rather than the compiler’s — transform only what is specific to economic modelling, and delegate everything else.

Pyomo, one of the algebraic modeling languages named earlier, is embedded in Python for the same reason, which suggests the trade is not peculiar to this problem class.

3.4 The ModelFlow equation DSL

A ModelFlow model is written as economic equations rather than as low-level Python array operations.

A simplified equation might be:

```
| FRML <> Y = C + I + 0.8 * Y( -1) $
```

The expression

```
| C + I + 0.8 * Y( -1)
```

is deliberately close to legal Python syntax.

Normal arithmetic, parentheses, operator precedence, function calls and nested expressions can therefore be delegated to Python. ModelFlow mainly has to understand constructs that have a special model meaning.

For example:

```
| Y( -1)
```

is syntactically a legal Python function call, but ModelFlow interprets it as variable *Y* lagged one period.

This allows ModelFlow to avoid implementing a complete expression language of its own.

3.4.1 Where the notation comes from

The notation is inherited rather than invented. *FRML* is TSP's keyword for a formula, and TSP is where it reaches ModelFlow from: the ADAM equations that the first generation of this architecture translated were TSP equations, as described in [Historical background](#).

TSP is old enough to be an ancestor of most of the field. Development began at MIT in 1965–1966 and continued at Berkeley, under Robert E. Hall, and by the time of Drud's survey it was running at over five hundred sites, among the most widely installed systems there. Its descendants are in use today: MicroTSP, written by David Lilien and first released in 1981, was rewritten for Windows and released as **EViews** in 1994 [Renfro, 2004].

The family resemblance is easy to see, and it runs wider than one lineage. A behavioural equation in EViews, in Dynare's *.mod* notation, in WinSolve and in the ModelFlow DSL is the same line of algebra with a different keyword in front of it and a different convention for writing a lag. That is not imitation. It is what a notation converges on once the target is fixed as time series models and the aim is to keep the specification close to the economics. It is also what makes the preprocessors described further down tractable: the equation bodies largely carry over, and the work is in the conventions surrounding them.

Individual equations like this one are, however, rarely written by hand in large models. They are usually *produced* — by the higher-level DSL, by an import preprocessor, or by normalization, each taken up in the sections that follow. Validation and token rewriting, further down the chain, operate on the equations that those earlier stages generate.

3.5 Higher-level model specification DSL

Large models often contain many equations that have the same economic structure but differ across dimensions such as sectors, ages, commodities, countries or sexes. Writing every expanded equation separately would be verbose and could hide the economic structure.

ModelFlow therefore has a higher-level specification DSL with constructs including **lists**, **do**, **doable**, **sum** and **tags**.

A list defines a model dimension. Each statement is terminated by **\$**:

```
list ages = ages : age_0 * age_101 $
list sexes = sexes : female , male $
```

The `*` in `age_0 * age_101` is a range operator: a common alphabetic prefix followed by a numeric range expands to the whole sequence `AGE_0, AGE_1, ... AGE_101`.

A single `list` statement can also define several parallel sublists, separated by `/`. The sublists are aligned element by element, so one loop can substitute matching values from each:

```
list banklist = bank      : Danske , Nordea , Jyske /
                  country : DK      , SE      , DK      $
```

An equation can then describe an entire family of equations:

```
doable POP_PRIMO__{SEXES}__{AGES}
      = POP__{SEXES}__{AGES_BEFORE}(-1) $
```

Summation can also be expressed directly at the model-specification level:

```
FRML <> MIGRATION__TOTAL =
      SUM(AGES, SUM(SEXES, MIGRATION__{SEXES}__{AGES})) $
```

The higher-level DSL is not merely shorthand. It expresses the dimensional and structural logic of the model more directly. The two levels are distinct in purpose: the higher-level DSL expresses dimensions, repeated structures, sums, tags and model transformations, while the equation DSL of the previous section expresses individual economic equations in Python-compatible notation. Expansion turns the first into the second.

3.6 Preprocessors and models from other systems

A model does not have to originate in the ModelFlow DSL. Source-specific preprocessors translate models from other modelling systems into the common ModelFlow representation.

Read the table as a list of models that have been onboarded, not as a claim of compatibility. **The preprocessors are not reimplementations of EViews, Dynare, GAMS or Excel.** They do not attempt to cover everything those systems can express. Each was written to bring in a particular model, and handles the constructs that model actually uses.

Source	Module	Preprocessor responsibility
EViews	<code>modelgrabwf2.py</code>	Read <code>.wf1/.wf2</code> workfiles; translate equations, data and metadata
World Bank macro models	<code>modelmacrograb.py</code>	Translate the specification through <code>modelnormalize</code>
Dynare	<code>model_dynare.py</code>	Read expanded <code>.mod</code> output; map syntax and timing conventions
GAMS / GDX	<code>modelgrabgdx.py</code>	Read GDX files via <code>gams.transfer</code> ; extract time-indexed variables and scenarios
Excel	<code>model_Excel.py</code>	Interpret workbook-based model specifications

Not every import has to become a library module. A model that is translated once, rather than repeatedly, can perfectly well be handled by a one-off script whose output is a finished ModelFlow model. The FRB/US model is the notable example: the Federal Reserve distributes its equations in XML, and the translation was done as a separate exercise, leaving behind a stored model that the example notebooks simply load. Only sources that are imported often enough to be worth generalizing earn a place in the table above.

Either way the effect is the same, and it is the point of the arrangement: source-specific knowledge is separated from the downstream model machinery. The source system holds the original specification; the preprocessor understands its syntax and conventions; the ModelFlow representation provides a common equation and model form; and the ModelFlow runtime provides common solving, simulation, analysis and reporting.

Once translated, models from different sources can use the same ModelFlow facilities for solving, scenarios, attribution, visualization and reporting.

3.6.1 Onboarding a model, not supporting a system

This is a deliberate choice about scope. The alternative is a trap.

General compatibility means committing to the whole of another system’s semantics. Every option, every estimation command, every corner of its syntax. Most of it no particular model uses. The obligation is unbounded, and it grows with every release of the other system.

Onboarding starts from the other end. The question is not “what can EViews express?” but “what does *this* model need?” That has a finite answer.

It can also be checked. The imported model is solved, and its results compared against the original system’s — equation by equation where necessary. A model that reproduces its source is onboarded, whatever fraction of the source language was involved.

The practical consequences are worth stating plainly:

- A model from a listed source is **not guaranteed to import**. It may use constructs no earlier model needed.
- The gap is then usually one specific construct. Extending the preprocessor to cover it is a bounded task.
- Coverage grows model by model, driven by demand rather than by the source system’s feature list.

The offer is not “run your EViews models in Python”. It is that an onboarded model gains the whole ModelFlow apparatus — graph analysis, attribution, the solvers, the reporting — whatever notation it was written in.

3.7 Normalization

Expanded equations are not yet in a form that can be evaluated. An equation may have an expression on the left-hand side, it may be implicit, and it may use economic transformation functions that have no Python equivalent.

Normalization (`modelnormalize.py`) closes this gap. It transforms

$$g(y, x) = f(y, x) \implies y = F(x, z) \quad (3.1)$$

so that each equation assigns exactly one endogenous variable. This is the step that makes the rest of the chain possible: the graph analysis needs a single left-hand-side variable per equation to define its nodes, and Gauss-Seidel needs an assignment it can evaluate.

Normalization has three parts.

3.7.1 Preprocessing of economic functions

Transformation functions familiar from econometric practice are expanded into elementary arithmetic before anything else happens:

Written	Expanded to
DIFF(X)	$((X) - (X(-1)))$
DLOG(X)	DIFF(LOG(X)), then expanded again
PCT(X)	$(100 * ((X) / (X(-1))) - 1)$
PCY(X)	as PCT, but against $X(-4)$
MOVAVG(X, n)	$((X + X(-1) + \dots) / n.0)$
LOGIT(X)	$(\text{LOG}(X / (1.0 - X)))$

$D(\dots)$ is accepted as an alias for $\text{DIFF}(\dots)$. A separate step, `fixleads`, rewrites a lead written as $X(1)$ into the canonical $X(+1)$ so that leads and lags are distinguishable later.

3.7.2 Symbolic isolation of the endogenous variable

Most equations need no isolation at all. If the left-hand side is already a single variable — the ordinary case $Y = f(X)$ — the equation is used as it stands, and no symbolic algebra is invoked.

Applying one of the preprocessing functions above to the endogenous variable itself does not change that in substance. $\text{DIFF}(Y) = f(X)$ unfolds to $(Y) - (Y(-1)) = f(X)$, and isolating Y is just moving $Y(-1)$ across the equals sign; $\text{DLOG}(Y) = f(X)$ unfolds to a difference of logs, and isolating Y is the same move. Each of these transformation functions has one fixed, known inverse, so in general DIFF and DLOG on the left-hand side are handled by shuffling terms, not by solving anything.

Genuine symbolic work is needed only when the left-hand side is not one of these recognized shapes — a hand-written multi-term expression $(A - C = B)$ — or when the modeller asks to normalize for a variable that sits on the right-hand side, where the coefficient or context is not fixed in advance. Lagged terms are first pasted into atomic symbols ($X(-1)$ becomes a single name ending in `__LAG`) so that SymPy treats them as constants rather than as function calls, and the result is unpasted afterwards. `LOG` and `EXP` are lowercased to match SymPy’s names, and a *clash* dictionary maps every model variable to an explicit symbol so that names such as `E`, `I`, `S` or `beta` are not silently reinterpreted as SymPy built-ins.

Equations that are genuinely implicit are handled differently: instead of being solved, they are turned into a residual equation

```
| Y__RES = ( rhs ) - ( lhs )
```

which the Newton solvers can use directly. [Model structure and solution](#) explains why the distinction between a normalized row and a `__RES` row matters for the Jacobian.

That some equations resist normalization is a known feature of econometric models rather than an awkwardness of this implementation. Don [1990] makes the point that a good normalization does not always exist — even where a bad one does — and treats the handling of implicit equations, whether individually noninvertible equations or system equilibrium conditions, as a problem to be solved rather than avoided. The residual form is how that is done here.

3.7.3 Generation of equation variants

The operational variants described in the next section are **not** produced by symbolic algebra. They are textual constructions — the add-factor term is appended, the exogenization switch is wrapped around the right-hand side, and the reverse equation is written directly. For an equation whose left-hand side is already a single variable, the whole set is assembled by string composition:

```
| normalized    Y = (f(X) + Y_A)
| fixable       Y = (f(X) + Y_A) * (1 -Y_D) + Y_X * Y_D
| add factor    Y_A = (Y) - (f(X))
| fitted       Y_FITTED = f(X)
```

This is worth stating plainly because it is what makes the variants cheap and predictable. No equation is re-derived, nothing depends on SymPy succeeding, and the generated forms are recognizably the modeller’s own right-hand side.

Where isolation is needed at all, the same shuffling continues to cover it. For $\text{DIFF}(Y) = f(X)$ with an add factor requested, the add-factor term is inserted before the equation is rearranged, and both variants fall out of the same move: $Y = Y(-1) + f(X) + Y_A$, and $Y_A = (Y) - (Y(-1)) - f(X)$. Nothing here needs more than moving terms across the equals sign — only the hand-written or right-hand-side-variable cases mentioned above call for SymPy to do work beyond that.

Normalization therefore emits, from one authoritative specification:

- the **normalized** equation, optionally with an add factor appended;
- the **add-factor** equation, for the reverse calculation;
- a **fixable** form, which embeds the exogenization switch;
- a **fitted** form, without add factor or exogenization, useful for estimation and for tracking historical fit.

3.8 Tags and generated model transformations

Tags allow information about how an equation should be treated to accompany the economic equation without putting operational details into the mathematical expression itself. Preprocessing uses them to select which of the normalization variants are generated for a given equation.

Important applications include add factors, exogenization, reverse calculation of add factors, and other model-specific transformations.

3.8.1 Add factors

Consider the authoritative economic equation:

$$Y = f(X)$$

For simulation, preprocessing can generate an equation with an add factor:

$$Y = f(X) + Y_A$$

The add factor allows the equation to reproduce a historical value, baseline path, judgemental adjustment or imposed scenario while retaining the underlying behavioural relationship. The suffix is configurable; `_A` is the default.

3.8.2 Exogenization

Normally Y is endogenous: the model calculates it. For some simulations, however, the modeller may want to impose a path for Y . The variable is then **exogenized**.

ModelFlow implements this without a second model. A *fixable* equation carries the switch in the equation itself:

$$Y = (f(X) + Y_A) * (1 - Y_D) + Y_X * Y_D$$

Y_D is a dummy: where it is 0 the equation behaves normally, and where it is 1 the value of Y is taken from the exogenous series Y_X . Exogenization therefore becomes a matter of setting data, and it can differ from period to period without regenerating the model.

3.8.3 Reverse calculation of add factors

If Y is given, the same economic relationship can be reversed to calculate the add factor required to make the equation hold:

$$Y_A = Y - f(X)$$

As described under normalization, this equation is not written separately by the modeller: it is generated alongside the normalized one, in the ordinary case simply as $Y_A = (Y) - (f(X))$. ModelFlow can collect these equations into a **separate reverse model**.

Model variant	Given	Calculated	Equation
Normal simulation	Y_A and explanatory variables	Endogenous Y	$Y = f(X) + Y_A$
Reverse/add-factor model	Exogenized Y and explanatory variables	Required Y_A	$Y_A = Y - f(X)$

This is useful when constructing a baseline or imposing a scenario path. The modeller can:

1. exogenize selected variables;
2. provide their desired paths;
3. run the reverse model;
4. calculate the required add factors;
5. return to the normal model with those add factors.

The normal and reverse models are generated from the **same authoritative economic specification**, so no hand-written second version of the model has to be maintained.

3.9 Validate before rewriting

Syntax checking is performed **before ModelFlow rewrites model tokens into implementation-level Python**.

Python's `ast` module can be used because the expression part of the ModelFlow DSL is intentionally Python-compatible:

```
import ast
ast.parse(expression)
```

The AST is used here for **validation and error localization**, not as the main transformation mechanism.

Suppose a modeller writes:

$$Y = C + (I * + 0.8 * Y(-1))$$

Because syntax validation occurs before rewriting, ModelFlow can point to the error in the original economic equation. The checker parses the expression **exactly as written** — without stripping blanks or newlines — precisely so that the line and column numbers reported by Python refer to the modeller's own text. It then reproduces the offending line with a caret under the reported column:

```
Syntax error at line 1, column 19: '(' was never closed
Y = C + (I * + 0.8 * Y( -1)
                ^
```

If variables had already been transformed into internal array references and indices, the syntax error could instead point into generated Python source that the modeller never wrote.

Thus Python's AST acts as a front-end syntax checker for the ModelFlow DSL.

3.10 Regex and token rewriting

After syntax validation, ModelFlow performs its domain-specific substitutions.

Regular expressions are appropriate because they are **not used to parse the complete expression grammar**. They identify only constructs for which ModelFlow has a special interpretation.

To see what the substitution produces, it helps to know what the model runs on. A ModelFlow model holds its data in a pandas DataFrame with periods as rows and variables as columns. For solving, that DataFrame is handed over as a plain two-dimensional NumPy array, `values`. Every model variable therefore has a fixed **column number**, and the current period is a **row number**, `row`.

A lag or lead is then nothing more than an offset on the row index. If `Y` happens to be column 42:

```
Y          -> values[row, 42]
Y( -1)     -> values[row -1, 42]
Y(+1)      -> values[row+1, 42]
```

The transformation is therefore mainly:

```
model token -> implementation token
```

Python remains responsible for the surrounding expression. This is also why lags are essentially free: no shifting or copying of series takes place during solving, only integer arithmetic on a row index.

3.10.1 Why the NumPy array rather than the DataFrame

Nothing forces this choice. The generated code could just as well address the DataFrame directly by label, which would be far more readable:

```
df.loc[period, 'Y'] = df.loc[period, 'C'] + df.loc[period, 'I'] + 0.8 *
df.loc[prev_period, 'Y']
```

The problem is *when* the names are resolved. Every one of those labels has to be looked up in the index and in the column index — a hash lookup, a check, and the construction of a scalar object — and pandas does that work **on every single access, every time the line is executed**. A model equation is not executed once: it is executed for every iteration, of every period, of every scenario. In a model of a few thousand equations solved over a few hundred periods, that is easily hundreds of millions of label lookups, all of them re-deriving an answer that never changes.

Translating the names into positions moves that work to transpile time. `Y` is looked up **once**, while the source is being generated, and the answer — column 42 — is baked into the generated text as an integer constant. At solve time nothing is resolved at all:

	Label access on the DataFrame	Position access on the array
Y resolved to a location	On every evaluation	Once, when the code is generated
Cost per access	Hash lookup, bounds check, scalar boxing	Pointer arithmetic on a float array
What varies at solve time	The name (constant) and the period	Only <code>row</code> , an integer

The saving is large — this is one of the main reasons ModelFlow can solve realistically sized models at acceptable speed — and it comes at no cost in expressiveness, since the modeller never sees the generated code anyway. The DataFrame is not abandoned: it remains the model’s data structure, and results are handed back as a DataFrame. It is simply stepped out of for the duration of the solve, via `.values`, which exposes the underlying block of floats without copying it.

There is a second consequence, taken up in the next section. A two-dimensional array of floats is exactly the kind of object Numba can compile; a pandas DataFrame is not. Resolving names to positions is therefore also what makes the just-in-time path possible at all.

3.10.2 A variant: the stuffed solution vector

Position access can in principle be taken one step further. Rather than indexing into the full two-dimensional array, a solver can gather the values a period actually needs — the current value of each variable, and those lags the equations really use — into a single compact one-dimensional vector, iterate on that, and write only the endogenous results back afterwards. Each variable and lag then has a fixed offset in that vector, and the whole working set is contiguous in memory.

ModelFlow has one Gauss-Seidel solver built this way, but it is an experiment rather than the normal path: the ordinary solvers address the two-dimensional array directly, as above. The idea is older than ModelFlow — it comes from PCIM, where it was a necessity rather than an option, and the [historical background](#) describes why.

3.11 Outsourcing the language chain to Python

After ModelFlow-specific substitutions, the result is legal Python source.

Python handles constructs such as:

```
A + B * C
(A + B) * C
A ** B
func(A, B)
-A
A / (B + C)
```

ModelFlow therefore does not have to duplicate Python’s rules for precedence, associativity, parentheses, numeric literals, function calls, unary operators, or nested expressions.

This strategy is not peculiar to ModelFlow. Generating code in a general-purpose language and letting its compiler do the rest is a well-established way of building modelling systems: Dynare translates its `.mod` specifications into MATLAB/Octave, and the approach goes back at least to the mainframe era. Only the source notation and the target language differ; the [historical background](#) discusses the parallel.

The generated source is text, so it can simply be compiled and executed:

```
code = compile(generated_source, filename, "exec")
exec(code, namespace)
```

The equations of the model become the body of a generated function, and solving the model means calling that function repeatedly.

3.11.1 Just-in-time compilation

Because the output is source text rather than an in-memory data structure, ModelFlow gets a second compiler for free. With the `ljit` option, the same generator emits the same equations decorated for Numba:

```
from numba import jit

@jit("(f8[:,:],f8[:,:],i8,f8)", fastmath=True, nopython=True)
def los(values, outvalues, row, alfa):
    ...
```

The explicit signature — two float arrays, an integer row and a float damping factor — lets Numba compile ahead of the first call instead of specializing on the first argument types it sees.

That signature is only writable because of the decision described under token rewriting. Numba’s `nopython` mode compiles a restricted set of types, and a two-dimensional array of floats indexed by integers is squarely within it, while a pandas DataFrame indexed by labels is not. Having already resolved every variable name to a column number, the generated equations contain nothing but arithmetic and integer indexing — so the same text that Python can `exec` can also be handed to Numba and compiled to machine code, with no separate code path for the two cases.

Two practical points follow from the size of real models. First, the equations are emitted in **chunks** (30 by default) wrapped by a master function, because a single function with many thousands of statements is slow for Numba to compile. Second, the generated source is written to a file under `modelsource/` and the compiled functions are cached on the model instance, so the compilation cost is paid once rather than on every solve.

The division of labour is:

Layer	Responsibility
ModelFlow DSL	Economic-model notation
Higher-level DSL	Dimensions, repeated structure, sums, tags
Normalization	Isolate the endogenous variable; generate equation variants
Python AST	Early syntax validation and error localization
Regex/token rewriting	Translation of ModelFlow-specific constructs
Python parser/compiler	General expression grammar and compilation
exec	Execution of generated model code
Numba (optional)	Machine-code compilation of the same generated source
ModelFlow solvers	Next stage: evaluation and solution; see Model structure and solution

3.12 Runtime error localization

Validating before rewriting keeps *syntax* errors anchored to the equation the modeller wrote. The same problem arises again at solution time, and for the same reason: the code that fails is generated code. A logarithm of a negative number or a division by zero raises an exception somewhere inside a machine-written function of several thousand lines.

ModelFlow solves this by generating the error handling along with the equations. Each block of generated code is wrapped so that an exception reports the line number at which it occurred:

```
except:
    errorfunkt(values, sys.exc_info()[2].tb_lineno, overhead=..., overeq=...)
```

Because the generator knows how many lines of preamble it emitted (`overhead`) and how many equations preceded the current chunk (`overeq`), the Python line number can be mapped back to a position in the model’s solve order — and therefore to a specific economic equation.

The handler then reports the failing equation, and it snapshots the entire data array as a DataFrame on the model (`.errdump`, copied to `.lastdf`) so the modeller can inspect the state of every variable at the moment of failure. It also inspects the failing equation for `LOG(...)` calls and

evaluates their arguments, since a non-positive argument to a logarithm is the single most common cause.

The design principle is the same at both ends of the chain: generated code is an implementation detail, and the modeller should never be asked to debug it.

3.13 A worked example: specification to executable code

The stages above are easier to see in one equation carried from specification to executable code. Take a consumption function specified over a list of household groups.

Specification. A list defines the dimension, and one `doable` describes the whole family of equations:

```
list groups = groups : young , old $

doable DIFF(C_{GROUPS}) = 0.5 * DIFF(YD_{GROUPS})
                        + 0.3 * (YD_{GROUPS}(-1) - C_{GROUPS}(-1)) $
```

Expansion. The higher-level DSL produces one equation per group. Following `C__YOUNG`:

```
FRML <> DIFF(C__YOUNG) = 0.5 * DIFF(YD__YOUNG)
                        + 0.3 * (YD__YOUNG(-1) - C__YOUNG(-1)) $
```

Preprocessing. `DIFF` is expanded into elementary arithmetic:

```
((C__YOUNG) - (C__YOUNG(-1)))
= 0.5 * ((YD__YOUNG) - (YD__YOUNG(-1)))
+ 0.3 * (YD__YOUNG(-1) - C__YOUNG(-1))
```

Normalization. The left-hand side is `DIFF(C__YOUNG)`, one of the recognized shapes, so isolating `C__YOUNG` is a shuffle: the add-factor term is inserted, and `C__YOUNG(-1)` is moved across the equals sign:

```
C__YOUNG = C__YOUNG(-1)
          + 0.5 * ((YD__YOUNG) - (YD__YOUNG(-1)))
          + 0.3 * (YD__YOUNG(-1) - C__YOUNG(-1))
          + C__YOUNG_A
```

The reverse equation falls out of the same move, this time isolating `C__YOUNG_A`. Had the specification been written directly as `C__YOUNG = ...`, with no `DIFF` on the left, both forms would have been assembled textually in exactly the same way:

```
C__YOUNG_A = ((C__YOUNG) - (C__YOUNG(-1)))
              - 0.5 * ((YD__YOUNG) - (YD__YOUNG(-1)))
              - 0.3 * (YD__YOUNG(-1) - C__YOUNG(-1))
```

If the equation is also made fixable, the normalized form is wrapped with the exogenization switch described under tags, `... * (1 - C__YOUNG_D) + C__YOUNG_X * C__YOUNG_D` — again assembled textually, not derived.

Validation. The normalized equation is parsed with `ast.parse` as written. Any syntax error is reported against this text — which the modeller can still recognize — rather than against the array code generated next.

Token rewriting. Each variable becomes a column index and each lag a row offset. If `C__YOUNG` is column 3, `YD__YOUNG` column 7 and `C__YOUNG_A` column 11:

```

values[row,3] = (values[row -1,3]
                + 0.5 * ((values[row,7]) - (values[row -1,7])))
                + 0.3 * (values[row -1,7] - values[row -1,3])
                + values[row,11])

```

Execution. This line is one statement in a generated function, compiled with `compile/exec` — or, with `ljit`, emitted into a Numba-decorated chunk and compiled to machine code. Gauss-Seidel solves the model by calling that function repeatedly.

The continuation of this example—its graph edge, symbolic derivative and Jacobian row—is given in [Model structure and solution](#).

3.14 Layered view of the translation pipeline

Layer	Main function
External model formats	EViews, Dynare, GAMS/GDX, Excel and other sources
Preprocessors	Translate source-specific representations into ModelFlow
Higher-level DSL	Lists, <code>do</code> , <code>doable</code> , <code>sum</code> , tags and dimensional expansion
Normalization	Expand economic functions; isolate the endogenous variable
Generated model variants	Add factors, exogenization, reverse and fitted equations
Equation DSL	Compact Python-compatible economic equations
Validation	Python AST checking against the original equation
Translation	Regex and ModelFlow-specific token substitution
Python toolchain	General parsing, compilation and execution
Numba (optional)	Machine-code compilation of the generated source
Runtime diagnostics	Map execution errors back to the failing economic equation

The output of this pipeline is an executable system of model equations. The next stage is structural analysis and numerical solution, described in [Model structure and solution](#).

Chapter 4

ModelFlow: Model Structure and Solution

4.1 Summary

Once ModelFlow has translated the economic specification into executable equations, the next question is how those equations are ordered and solved efficiently.

This chapter starts from the model's directed dependency graph. It uses strongly connected components to separate **prolog**, **simultaneous core** and **epilog**, and an approximate feedback vertex set to expose a smaller set of variables that drives the remaining cycles. That structure is then used by the numerical solvers: conventional Gauss-Seidel for many normalized macroeconomic models, Newton methods for harder simultaneous or implicit systems, reduced Newton on feedback variables, and stacked Newton when endogenous leads couple periods together.

The preceding chapter, [From model specification to executable code](#), explains how the equations reach executable form. The following chapter, [Local stability and model dynamics](#), reuses the Jacobian machinery to analyse the model rather than solve it.

4.2 Model structure as a directed graph

A ModelFlow model can also be represented as a **directed dependency graph**. Each model variable is a node. For an equation with left-hand-side variable Y , every model variable on the right-hand side has a directed edge pointing to Y :

| RHS variable - - - - -> LHS variable

The graph therefore describes the **computational dependency structure** implied by the normalized model equations. When the equations themselves have a structural economic interpretation, this ordering can also be read as causal in that model-specification sense; it should not be confused with a causal graph established independently by statistical identification. This is one of the reasons normalization matters: without a single left-hand-side variable per equation, the nodes of this graph would not be well defined.

4.2.1 Dependencies and solving order

If the dependency graph contains no cycles, equations can be evaluated in topological order: up-stream variables are calculated before variables that depend on them. Cycles identify feedback and simultaneous relationships.

ModelFlow can use this graph structure to establish an efficient solving order.

4.2.2 Prolog, core and epilog

The ordered model is divided into three computational parts:

Part	Graph interpretation	Role in solution
Prolog	Variables upstream of the simultaneous core	Calculated once before iteration
Core	Simultaneously interdependent variables	Recalculated during iterative solution
Epilog	Variables downstream of the simultaneous core	Calculated after the core has converged

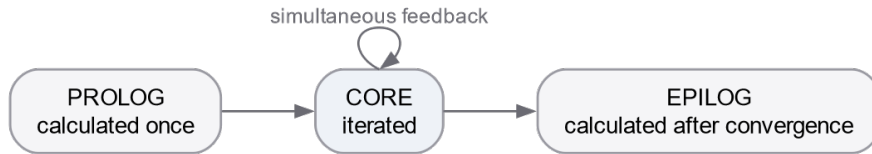


Figure 4.1: Prolog, core and epilog.

This decomposition avoids repeatedly evaluating equations outside the simultaneous part. The prolog is evaluated before iteration, only the core must be repeatedly evaluated, and the epilog is evaluated after convergence.

4.2.3 Strongly connected components

A **strongly connected component** is a set of nodes in which every node is reachable from every other node. In model terms, it represents variables connected through feedback.

A model can contain several simultaneous strongly connected components. For ModelFlow’s computational representation, these are collected into one **core**. Equations that can be ordered before the core form the prolog; equations downstream of the core form the epilog.

Thus the overall computational structure remains:

```
| prolog -> core -> epilog
```

even when several strongly connected components occur inside the core.

4.2.4 Feedback variables and the feedback vertex set

Even inside the core, it is often unnecessary to treat every endogenous variable as an independent feedback variable.

A **feedback vertex set** is a set of nodes which, if removed from the directed graph, breaks all directed cycles. If the feedback variables are held fixed at the beginning of an iteration, the remaining equations can be evaluated in an acyclic computational order. The feedback variables are then updated and convergence tested.

Finding the **minimum feedback vertex set** is computationally hard for a general directed graph. It is not necessary, however, to find the mathematically smallest set to obtain most of the computational benefit. A good approximation can identify a relatively small feedback set and reduce the effective iterative problem.

None of this is new. Don [1990] sets out the argument for econometric models directly: a large sparse system can be solved efficiently if the equations are ordered so that the set of feedback variables is small, the reordering problem is the standard graph-theoretic one of finding a minimal feedback vertex set, that problem is NP-hard, and a heuristic in polynomial time is therefore used to produce small — though not necessarily minimal — feedback sets. He also notes what is claimed here: that the same reordering improves Gauss-Seidel convergence, not only Newton.

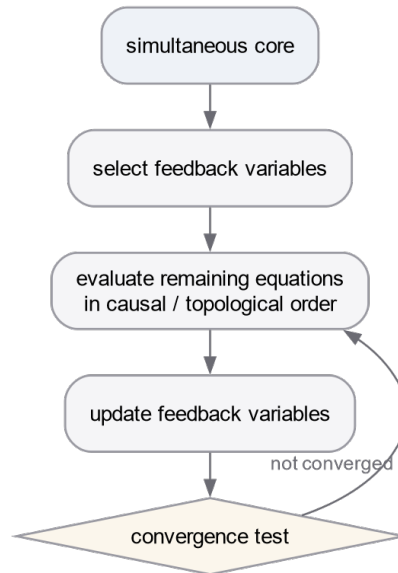


Figure 4.2: One pass over the feedback variables.

Without feedback reduction	With feedback-set reduction
Large simultaneous core repeatedly evaluated	Smaller feedback set drives iteration
Less use of causal ordering within the core	Remaining equations use computational order
More work per iteration	Potentially substantially less work per iteration

This splits the core into a small set of feedback equations and a larger acyclic remainder (**daglist**). The same split is just as useful for Newton, where it reduces the dimension of the linear system rather than the cost of an iteration; the Newton section returns to it.

4.2.5 Graph analysis in ModelFlow

The dependency graph serves several purposes:

Graph information	ModelFlow use
Directed RHS-to-LHS edges	Represent equation dependencies
Topological ordering	Establish causal/computational evaluation order
Strongly connected components	Identify simultaneous feedback structure
Prolog/core/epilog decomposition	Avoid unnecessary repeated evaluation
Approximate feedback vertex set	Reduce the effective iterative problem
Lead/lag relationships	Characterize dynamic dependencies across periods

Graph analysis complements the numerical solvers. For Gauss-Seidel it provides equation ordering and feedback reduction. For Newton methods it exposes structural dependencies, reduces the system that has to be factorized, and helps explain the sparsity pattern of the Jacobian.

The directed graph is therefore an important intermediate representation connecting **model specification, causal structure and numerical solution**.

4.3 Gauss-Seidel solution

For many economic models, the natural solution method is an iterative **Gauss-Seidel** solution.

The equations are arranged in computational order and evaluated repeatedly. As soon as a new value of an endogenous variable has been calculated, that updated value is available to subsequent equations in the same iteration.

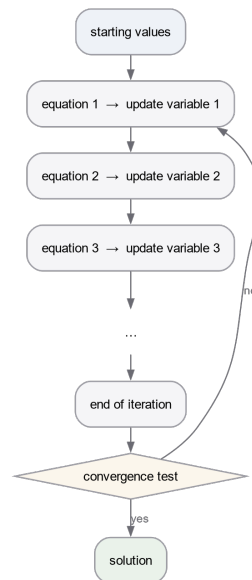


Figure 4.3: A Gauss-Seidel sweep.

Conceptually:

For a dynamic model without leaded endogenous variables, periods can normally be solved sequentially:



Figure 4.4: Sequential solution, period by period.

Lagged variables are already known from earlier periods, so they fit naturally into this solution strategy.

The update can be damped. Instead of accepting a newly calculated value outright, the solver can move only part of the way towards it, which stabilizes oscillating feedback loops at the cost of more iterations.

Gauss-Seidel has important practical advantages:

- it is simple;
- it exploits the equation structure directly;
- it does not require construction or factorization of a Jacobian;
- and it works well for many traditionally specified macroeconomic models.

For more difficult nonlinear systems, implicit equations, or models where stronger simultaneous solution is required, ModelFlow can use Newton methods.

4.4 Newton solution and the Jacobian

Newton solution requires the **Jacobian matrix**: the matrix of derivatives of model equations with respect to endogenous variables.

ModelFlow does not require the modeller to supply this matrix manually.

Instead, ModelFlow creates a new model instance whose purpose is to calculate the relevant Jacobian coefficients.

The process is:

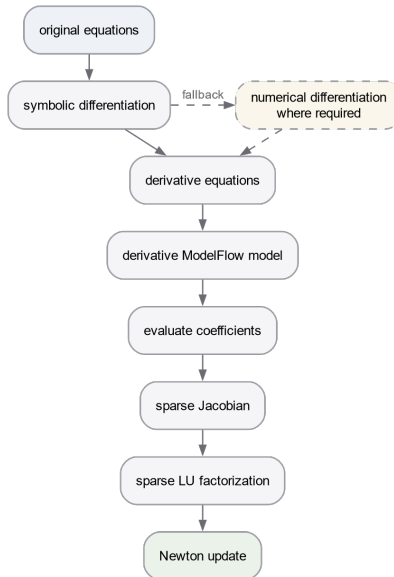


Figure 4.5: From equations to a Newton update.

4.4.1 Symbolic differentiation

ModelFlow first attempts to differentiate the original equations **symbolically**, using SymPy.

This reuses the machinery already described under normalization. The equation is handed to SymPy in the same way: `LOG(` is lowercased to `log(` because SymPy uses lower-case function names, and a clash dictionary protects model variables whose names collide with SymPy’s own symbols. The derivative is then taken with respect to each endogenous variable that actually occurs on the right-hand side.

The resulting derivative expressions are themselves equations. They can therefore be represented and evaluated using the same ModelFlow equation machinery — expansion, validation, token rewriting, `exec`, optionally Numba. A separate ModelFlow instance evaluates these derivative equations and produces the non-zero coefficients required by the Jacobian.

4.4.2 Numerical differentiation as fallback

Some expressions contain functions for which symbolic differentiation is unavailable or inconvenient — user-defined functions, interpolations, or constructs that SymPy cannot sensibly handle.

ModelFlow can then use **numerical differentiation as a fallback**, either for individual equations or, on request, for the model as a whole.

This combines the efficiency and precision of symbolic differentiation where possible with the generality of numerical differentiation where necessary.

4.4.3 Normalized and residual rows

The Jacobian must be built consistently with the form each equation actually takes. A normalized equation $y = F(y, x)$ has residual $F(y) - y$, so its row carries an additional -1 on the diagonal. An implicit equation, stored as $y_{\text{RES}} = G(y, x)$, has residual $G(y)$ and carries no such term.

ModelFlow keeps a per-equation mask recording which form each row has, and builds the Jacobian accordingly. This is not a detail: treating $F(y)$ itself as the residual of a normalized equation gives the Jacobian $\partial F / \partial y$ instead of $\partial F / \partial y - I$, and the iteration diverges.

4.4.4 Sparse Jacobian construction

Large macroeconomic models can contain thousands of endogenous variables, but an individual equation normally depends on only a small fraction of them.

The Jacobian is therefore highly sparse.

Jacobian element	Meaning	Sparse representation
$\partial F_i / \partial x_j \neq 0$	Equation i depends on endogenous variable j	Store row, column and coefficient
$\partial F_i / \partial x_j = 0$	No direct dependence	Structural zero; normally not stored

The derivative model calculates the coefficients, which are then **stuffed into the appropriate positions of a sparse matrix**.

ModelFlow can thereby retain knowledge of the economic equation structure while Python’s efficient sparse libraries handle storage and numerical linear algebra.

4.4.5 Sparse LU decomposition

At a Newton iteration the linearized system has the form:

$$J(x) \Delta x = -F(x) \quad (4.1)$$

The sparse Jacobian is factorized using an efficient sparse **LU decomposition**, and the factorization is used to obtain the Newton update:

$$x_{k+1} = x_k + \Delta x \quad (4.2)$$

4.4.6 Reusing the factorization

Rebuilding and refactorizing the Jacobian at every iteration is usually wasteful: in a mildly nonlinear model an old factorization remains a perfectly good approximation, and each reuse saves the dominant cost of the iteration.

ModelFlow makes this a single explicit policy rather than a scattering of heuristics. A `nonlin` option decides all refreshing: the factorization is carried forward across iterations and periods, refreshed on a fixed schedule (every N iterations), and refreshed additionally whenever the iteration appears to be diverging. A negative setting requests a fresh Jacobian before each new period as well. For a model solved over many periods with a repeated shock, this is the difference between factorizing thousands of times and factorizing a handful of times.

4.4.7 Newton on the feedback set

The feedback vertex set of the graph analysis is just as useful here, and for a sharper reason: it reduces the *dimension of the linear system*. This is the reduction Don [1990] describes — a system that is large but sparse can be solved by a Newton-type algorithm once it has been reduced to the low-dimensional problem in the feedback variables.

If the core is split into a small feedback set and an acyclic remainder, then the remainder is not a system to be solved at all — it can be evaluated exactly by a topological sweep, given the feedback values. Newton therefore only has to iterate on the feedback unknowns, and the linear algebra shrinks from the size of the core to the size of the feedback set.

ModelFlow offers several ways of obtaining that reduced Jacobian:

Reduced Jacobian	How it is obtained	When it fits
Schur complement	Exact reduction of the full Jacobian to the feedback unknowns	The exact Newton step, at reduced size
Finite differences	Perturb each feedback unknown and re-sweep the DAG	Captures coupling through the DAG chain without symbolic work
Direct	Differentiate the feedback equations only, iterating the inner solve	Nonlinear block Gauss-Seidel on the feedback block
Gauss (none)	Damped fixed-point step, no Jacobian at all	Robustness fallback for large or ill-conditioned feedback sets

The finite-difference variant is worth noting: it perturbs a feedback unknown, runs the acyclic sweep, and observes the effect on the feedback residuals. It therefore captures the *total* derivative

through the whole DAG chain, not merely the direct dependence between feedback equations — and it does so without differentiating anything symbolically.

4.4.8 Convergence

Gauss-Seidel and Newton share a relative convergence test: a variable is examined only if its absolute value is at least **absconv**, and it must then change by less than **relconv** between iterations.

The floor is necessary — otherwise a variable near zero would demand impossible relative precision — but it has a consequence worth stating plainly. A test on the *step* is blind to unknowns whose values sit below the floor. A saturated 0/1 policy switch, for example, can move between two small values without the test ever noticing, and the solver reports convergence on a system whose equations do not actually hold.

The Newton solvers therefore add an **absolute residual** test: convergence requires not only that the step be small, but that the equations be satisfied —

$$|G(y)| \leq \text{newton_absconv} \quad (\text{residual rows}) \quad (4.3)$$

$$|F(y) - y| \leq \text{relconv} \cdot \max(|y|, \text{absconv}) \quad (\text{normalized rows}) \quad (4.4)$$

Requiring both is what distinguishes “the iteration stopped moving” from “the model is solved”.

4.5 Stacked Newton solution for leaded endogenous variables

A dynamic model containing only contemporaneous and lagged endogenous variables can normally be solved one period at a time.

The situation changes when the model contains **leaded endogenous variables**.

An equation in period t may then depend on an endogenous variable in $t+1$. Future periods and current periods become mutually dependent, so sequential period-by-period solution is no longer sufficient.

ModelFlow can solve this by stacking all solution periods into one simultaneous system.

If the endogenous vector in one period is x_t , the stacked unknown vector is conceptually:

$$X = \begin{bmatrix} x_{t_0} \\ x_{t_1} \\ x_{t_2} \\ \vdots \\ x_{t_N} \end{bmatrix} \quad (4.5)$$

The Jacobian is correspondingly stacked across variables and periods.

A simplified block structure is:

Equation block	Variables $t-1$	Variables t	Variables $t+1$
Equations $t-1$	J	L	.
Equations t	B	J	L
Equations $t+1$	.	B	J

Here:

- J represents contemporaneous dependencies;
- B represents lagged dependencies;
- L represents leaded dependencies;
- . represents a structural zero block.

The resulting matrix can be extremely large, but it is also **very sparse**.

The solution chain becomes:

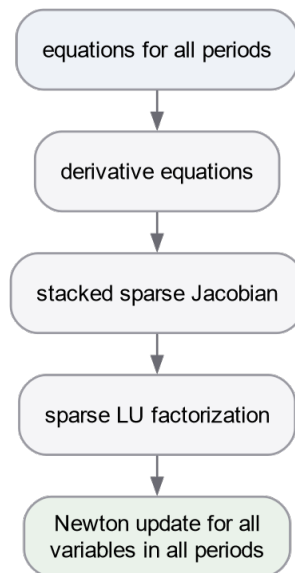


Figure 4.6: The stacked chain.

4.5.1 The stacked graph

The feedback reduction described for Newton extends to the stacked problem, but it needs a graph that does not exist on the model: the dependency graph over (period, variable) pairs.

ModelFlow constructs it from the structure of the stacked Jacobian itself. Each unknown becomes a node, and each non-zero derivative becomes an edge from `(t + lag, explanatory variable)` to `(t, endogenous variable)`. The result is decomposed exactly as in the graph analysis — strongly connected components in topological order, and an approximate minimum feedback set within each simultaneous block.

The payoff is that the acyclic sweeps may now run *across* periods. An equation whose inputs happen to be known, whether from an earlier period or a later one, is simply evaluated; only the stacked feedback unknowns are given to Newton. Models with leads are covered by exactly the same machinery as models without them, and the reduced system is a small fraction of the full stacked one.

This allows forward-looking models to be solved as one simultaneous intertemporal system while exploiting Python’s highly optimized sparse-matrix libraries.

4.6 Solver choice

The main solution approaches serve different model structures.

Feature	Gauss-Seidel	Newton	Newton on feedback set	Stacked Newton
Jacobian required	No	Yes	Reduced only	Yes, stacked
Basic operation	Repeated equation evaluation	Linearization and sparse linear solve	DAG sweep plus small linear solve	Linearization across all periods
Size of linear system	—	Endogenous variables in the core	Feedback variables only	Variables x periods
Sequential periods	Natural	Also period by period	Also period by period	No: all periods at once
Leaded endogenous variables	Not naturally suited	Not covered period by period	Not covered period by period	Handled by construction
Sparse matrix factorization	Not required	Sparse LU	Dense or Schur-complement reduced	Sparse LU
Derivative model	Not required	Symbolic with numerical fallback	Symbolic, finite-difference or none	Symbolic with numerical fallback
Typical strength	Simple and robust for many traditional models	Strong simultaneous solution for difficult nonlinear systems	Large cores with small feedback sets	Forward-looking models

Note that Newton is *not* inherently a whole-sample method. Where the model has no leads, the Newton solvers work period by period exactly as Gauss-Seidel does; stacking is required only when leaded endogenous variables make the periods mutually dependent.

ModelFlow can therefore choose a solution strategy appropriate to the structure of the model rather than imposing one numerical method on all models.

A graded hierarchy of this kind is long-standing practice rather than a novelty: Petersen and Sims [1987] describe Project LINK solved through a Jacobi, Gauss-Seidel and Newton hierarchy applied according to the difficulty of the system, on hardware two generations removed from anything discussed here. The reasoning survives the change of machine — Newton is more capable and more expensive, so it earns its place only where cheaper iteration fails.

The numerical methods themselves are not ModelFlow’s invention and are not described here in any depth. For the underlying theory — iterative and Newton-type solution of large nonlinear systems, ordering and decomposition, and the sparse techniques that make them practical — Pauletti [1997] is the standard treatment for macroeconomic models specifically, and covers most of what this chapter applies.

4.7 Worked example: from executable equation to graph and Jacobian

The worked example in [From model specification to executable code](#) ended with the normalized equation for `C__YOUNG` translated to array-indexed Python. The same equation also contributes to the dependency graph and, for Newton solution, to the Jacobian.

Differentiation. For Newton, SymPy differentiates the normalized equation with respect to the endogenous variables on its right-hand side. Here the only contemporaneous endogenous variable is `YD__YOUNG`:

$$\frac{\partial C_{\text{YOUNG}}}{\partial YD_{\text{YOUNG}}} = 0.5 \quad (4.6)$$

The lagged terms do not appear: within a period they are known constants, and they enter only the `B` blocks of the stacked Jacobian.

The Jacobian row. Because this is a normalized equation, its row carries the `-1` diagonal term described earlier. In the row for `C__YOUNG`:

```

column C__YOUNG   : -1.0
column YD__YOUNG  :  0.5
all other columns : structural zero, not stored

```

Two non-zeros in a row that is as wide as the model is large — which is why the Jacobian is stored sparsely, and why `YD__YOUNG ->C__YOUNG` is also exactly the edge this equation contributes to the dependency graph.

4.8 Solution architecture at a glance

Layer	Main function
Dependency graph	Directed RHS-to-LHS model dependencies
Graph decomposition	Topological ordering, strongly connected components, prolog/core/epilog
Feedback reduction	Approximate feedback vertex set and acyclic remainder
Gauss-Seidel	Repeated equation evaluation in computational order
Derivative model	Symbolic differentiation with numerical fallback
Newton	Sparse Jacobian, LU factorization and simultaneous update
Feedback-set Newton	DAG sweeps plus a smaller nonlinear system in feedback variables
Stacked Newton	Sparse simultaneous solution across periods when endogenous leads are present

The Jacobian produced for Newton also provides a local linearization of the model. That second use is the subject of [Local stability and model dynamics](#).

Chapter 5

ModelFlow: Local Stability and Model Dynamics

5.1 Summary

ModelFlow can reuse the derivative machinery built for Newton solution to study the local dynamics of a solved model. Contemporaneous effects are solved out, lagged endogenous effects are assembled into a companion matrix, and the eigenvalues of that matrix show whether small disturbances decay, persist, oscillate or grow.

The analysis is **local and period-specific**: it describes small perturbations around the point where the Jacobian is evaluated, not the nonlinear model everywhere. ModelFlow can also recompute the dominant root after removing individual equations, providing a diagnostic of which relationships carry an instability.

This chapter follows [Model structure and solution](#). The translation from economic specification to executable equations is described in [From model specification to executable code](#).

5.2 Linearization, the companion matrix and local stability

The derivative machinery used for Newton solution has a second use: it provides a local linearization of the model that can be analysed in its own right.

A Jacobian *is* a linearization. Once ModelFlow has evaluated the derivatives around a solved point, the resulting approximation can be used to ask a different question from the one posed by the solver: **is the model locally stable there, and if not, what is making it unstable?**

The construction starts from the same derivative model described in [Model structure and solution](#), but the output is analysed rather than used for a Newton step. The implementation is in `modelnewton.py` (the `newton_diff` class) and reached from `modelclass.py`.

5.2.1 From Jacobian to transition matrix

Take a model with n endogenous variables y , k exogenous variables x , maximum endogenous lag r and maximum exogenous lag s , where t is the time frame — year, quarter, day, second or another unit. Written out equation by equation, the whole model is

$$\begin{aligned} y_t^1 &= F^1(y_t^2 \dots y_t^n, y_{t-1}^1 \dots y_{t-r}^n, x_t^1 \dots x_t^k, \dots x_{t-s}^1 \dots x_{t-s}^k) \\ y_t^2 &= F^2(y_t^1 \dots y_t^n, y_{t-1}^1 \dots y_{t-r}^n, x_t^1 \dots x_t^k, \dots x_{t-s}^1 \dots x_{t-s}^k) \\ &\vdots \\ y_t^n &= F^n(y_t^1 \dots y_t^{n-1}, y_{t-1}^1 \dots y_{t-r}^n, x_t^1 \dots x_t^k, \dots x_{t-s}^1 \dots x_{t-s}^k) \end{aligned} \tag{5.1}$$

This is the **normalized** form, which is what the analysis below assumes: each endogenous variable is on the left hand side exactly once, and an unlagged endogenous variable never appears on the right hand side of its own equation. Collecting the variables of a period into the vectors $\mathbf{y}_t$ and $\mathbf{x}_t$, the same system is

$$\mathbf{y}_t = \mathbf{F}(\mathbf{y}_t \cdots \mathbf{y}_{t-r}, \mathbf{x}_t \cdots \mathbf{x}_{t-s}) \quad (5.2)$$

The derivative model produces the Jacobian split by lag, which gives three families of matrices:

$$A = \frac{\partial \mathbf{F}}{\partial \mathbf{y}_t^T}, \quad E_i = \frac{\partial \mathbf{F}}{\partial \mathbf{y}_{t-i}^T} \quad (i = 1 \dots r), \quad F_j = \frac{\partial \mathbf{F}}{\partial \mathbf{x}_{t-j}^T} \quad (j = 0 \dots s) \quad (5.3)$$

A holds the contemporaneous derivatives, E_i the lagged endogenous ones, F_j the exogenous ones. Linearizing around a solution, the response to a small perturbation is

$$\Delta \mathbf{y}_t = A \Delta \mathbf{y}_t + \sum_{i=1}^r E_i \Delta \mathbf{y}_{t-i} + \sum_{j=0}^s F_j \Delta \mathbf{x}_{t-j} \quad (5.4)$$

The contemporaneous term appears on both sides, so it must be resolved first:

$$(\mathbb{I} - A) \Delta \mathbf{y}_t = \sum_{i=1}^r E_i \Delta \mathbf{y}_{t-i} + \sum_{j=0}^s F_j \Delta \mathbf{x}_{t-j} \quad (5.5)$$

$$\Delta \mathbf{y}_t = \sum_{i=1}^r C_i \Delta \mathbf{y}_{t-i} + \dots, \quad C_i = (\mathbb{I} - A)^{-1} E_i \quad (5.6)$$

Inverting $\mathbb{I} - A$ is the same operation the Newton solvers perform, applied here to a different end: the simultaneous block is solved out, leaving a system in the lags alone.

The exogenous terms carry through, but they play no part in what follows. Stability is a question about how the system responds to its *own* past, so it is settled by the endogenous transition alone; the F_j describe how disturbances enter, not whether they die out.

5.2.2 The companion matrix

A system with r lags is a difference equation of order r , and stability is read off a *first-order* system. There is a standard way to get one: stack the lagged states into a single state vector

$$\mathbf{z}_t = \begin{bmatrix} \mathbf{y}_t \\ \mathbf{y}_{t-1} \\ \vdots \\ \mathbf{y}_{t-r+1} \end{bmatrix} \quad (5.7)$$

and write the whole system as one step, $\Delta \mathbf{z}_t = M \Delta \mathbf{z}_{t-1}$:

$$\Delta \mathbf{z}_t = \underbrace{\begin{bmatrix} C_1 & C_2 & \cdots & C_{r-1} & C_r \\ \mathbb{I} & 0 & \cdots & 0 & 0 \\ 0 & \mathbb{I} & \cdots & 0 & 0 \\ \vdots & & \ddots & & \vdots \\ 0 & 0 & \cdots & \mathbb{I} & 0 \end{bmatrix}}_{M = (\mathbb{I} - A)^{-1} \bar{E}} \Delta \mathbf{z}_{t-1} \quad (5.8)$$

M is the **companion matrix**. Its top block row carries the C_k ; everything below it is an identity block that simply shifts each lag down one place. ModelFlow assembles it exactly this way — the C_k horizontally stacked on top, a rectangular identity underneath.

5.2.3 Eigenvalues and local stability

One matrix now describes the whole dynamic system, and its eigenvalues characterize the behaviour:

Eigenvalues λ of M	Behaviour
all $ \lambda < 1$	The system converges — a disturbance dies out
at least one $ \lambda > 1$	The system amplifies — a disturbance grows
$ \lambda = 1$	On the boundary; perpetual, undamped movement
at least one λ has an imaginary part	The system oscillates — damped if all $ \lambda < 1$, amplifying if any $ \lambda > 1$

Magnitude and oscillation are separate questions: the moduli decide whether movement grows or dies away, the imaginary parts decide whether it is monotone or cyclical. The **largest modulus** is therefore the summary number, and a complex dominant root says the model cycles on its way there.

ModelFlow computes the eigenvalues with a dense solver, falling back to a sparse one for large systems, and plots them in polar coordinates — modulus as distance from the origin, argument as angle — so that the unit circle is the stability boundary and oscillatory roots are visible off the real axis. The calculation is carried out by a `newton_diff` object — obtained from a model with `get_newton()` — whose `get_eigenvalues()` builds the companion matrix per period; the eigenvalues and their eigenvectors are then available per period from `get_df_eigen_dict()`.

The next chapter, [Stability in practice](#), works through the five cases on a Samuelson multiplier-accelerator model — no amplification, explosion, exploding oscillations, perpetual oscillations and damped oscillations — showing each as a picture of where the roots sit relative to the unit circle.

Two properties of this matter in practice.

It is local, and therefore period-specific. The derivatives are evaluated at a particular solution in a particular period, so the answer describes the model *there*. ModelFlow computes the eigenvalues per period, and they can differ from one year to the next. A model can be stable in one part of the sample and not in another — which is itself a finding, and one that a single global statement about the model would hide.

It is a property of the linearization, not of the model everywhere. Local stability says what happens to small disturbances near the evaluation point. It is silent about large shocks and about behaviour far away.

5.2.4 Which equation is responsible

Knowing that a model amplifies is less useful than knowing *what* amplifies. ModelFlow can therefore delete the row and column belonging to one equation, recompute the eigenvalues, and repeat for every equation in turn — ranking them by how much their removal changes the largest modulus. `get_eigen_jackknife_abs_select(year)` returns that ranking, together with the undisturbed value for comparison.

An equation whose removal collapses the dominant root is carrying the instability. That is a pointer to the mechanism, since each equation is one endogenous variable and one behavioural relationship.

Two honest qualifications. It is expensive — the eigenvalues are recomputed once per equation, so the results are cached. And it is uninformative on small models, where deleting almost any equation breaks the single loop that drives everything; its value is on large models, where it can isolate the few equations that matter out of thousands. The name is borrowed loosely from jackknife resampling and is not an exact fit.

5.2.5 Application: macroprudential models

One possible - but untested - use of this feature is **macroprudential model**.

The reasoning is this. Macroprudential models are about feedback: credit conditions affect asset prices, asset prices affect balance sheets, balance sheets affect credit conditions. The policy question is often not the level of a forecast but whether such loops **amplify or damp** a shock, and whether that has changed over time.

That is the question the eigenvalues answer. A dominant root that has moved closer to the unit circle across the sample would say that the same shock now produces a larger and longer-lasting response — a statement about the system rather than about any single variable. A complex dominant root would say the response cycles. The jackknife would then point at the equations carrying it.

The attraction is that none of this requires a second model. The stability calculation is derived from the same equations, by the same derivative machinery, as the simulation — so there is no separate linearized model to build, calibrate and keep in step with the one actually being used.

The caveats from earlier in this section apply with some force here. The result is local and period-specific, and macroprudential concern is precisely with large shocks and non-linear behaviour, which a linearization around a solution does not describe. Whether the local measure says anything useful about the non-linear regime is exactly the question that would have to be settled by trying it.

Chapter 6

Stability in practice: a multiplier-accelerator model

Warning

This chapter is both unfinished and unpolished. Suggestions are very welcome. Terminology varies very much between fields of study.

This is the worked example belonging to [Local stability and dynamics](#), where the construction is derived in full. Here a Samuelson multiplier-accelerator model is run through the five classic cases, each one shown as the position of the roots relative to the unit circle.

6.1 The calculation in one paragraph

ModelFlow differentiates the model around a solved point. That gives the contemporaneous Jacobian A and one lagged Jacobian E_i for each lag $i = 1 \dots r$. Solving out the contemporaneous block gives $C_i = (\mathbb{I} - A)^{-1}E_i$, and stacking the lags into the state vector $\mathbf{z}_t = [\mathbf{y}_t, \mathbf{y}_{t-1}, \dots, \mathbf{y}_{t-r+1}]'$ turns the model into a single first order system

$$\Delta \mathbf{z}_t = M \Delta \mathbf{z}_{t-1}, \quad M = (\mathbb{I} - A)^{-1} \bar{E} \quad (6.1)$$

where M is the **companion matrix**: the C_i stacked horizontally on top, a rectangular identity underneath. Its eigenvalues λ - which can be complex numbers - answer the stability question:

- all $|\lambda| < 1$: the system converges, a disturbance dies out
- at least one $|\lambda| > 1$: the system amplifies, a disturbance grows
- $|\lambda| = 1$: on the boundary, perpetual undamped movement
- at least one λ with an imaginary part: the system oscillates - dampened if all $|\lambda| < 1$, amplifying if any $|\lambda| > 1$

The calculation is carried out by the `newton_diff` class in `modelnewton.py`. An instance is created from a model with `<modelclass>.get_newton()`; `.get_eigenvalues()` then builds the companion matrix and its eigenvalues for each period, and `.get_df_eigen_dict()` returns them as a dictionary where the key is the year and the value is a dataframe with the eigenvalues in the first row and the corresponding eigenvectors below.

The results below do not necessarily give economic meaning. They show how the stability of the system is reflected in the eigenvalues of the companion matrix.

6.2 The model

The model is written with the `%%Makemymodel` cell magic: lines starting with `>` are ModelFlow equations, the rest is documentation which is rendered together with them.

```
%%MakemyModel Mmma
**A Samuelson multiplier -accelerator model**

Output is consumption plus investment:
> gdp = consumption + investment

Consumption is a fraction `mul` of last years output - the multiplier:
> consumption = mul * gdp( -1)

Investment reacts to the change in consumption by the factor `acc` - the accelerator:
> investment = acc * (consumption - consumption( -1)) + exo_investment
```

A Samuelson multiplier-accelerator model

Output is consumption plus investment:

```
> gdp = consumption + investment
```

Consumption is a fraction mul of last years output - the multiplier:

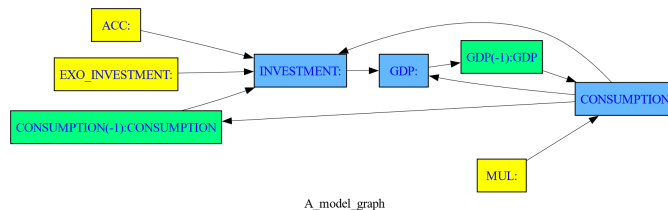
```
> consumption = mul * gdp( -1)
```

Investment reacts to the change in consumption by the factor acc - the accelerator:

```
> investment = acc * (consumption - consumption( -1)) + exo_investment
```

```
mma = Mmma.mmodel                                     # the solvable modelclass.model →
instance
mma.name = 'Accelerator multiplicator model'

mma.drawmodel(png=latex, des=False)
```



6.3 Data

We need some data to simulate. `get_df` generates a dataframe with the parameters MUL and ACC, and with exogenous investment.

```
def get_df(mul=0.9, acc=0.5, gdp=1000, im=200, years=100):
    'Creates input dataframe to the accelerator multiplier model'
    df = pd.DataFrame([[gdp]]*years, index=range(2018, 2018+years), columns=['GDP'])
    df.loc[:, 'MUL'] = mul
    df.loc[:, 'ACC'] = acc
    df.loc[:, 'CONSUMPTION'] = gdp*mul
    df.loc[:, 'EXO_INVESTMENT'] = im
    return df

base = get_df()
base.head()
```

	GDP	MUL	ACC	CONSUMPTION	EXO_INVESTMENT
2018	1000	0.9	0.5	900.0	200
2019	1000	0.9	0.5	900.0	200
2020	1000	0.9	0.5	900.0	200
2021	1000	0.9	0.5	900.0	200
2022	1000	0.9	0.5	900.0	200

6.4 A function which simulates the model and calculates the eigenvalues

This can be ignored if you are not interested in the Python implementation.

Note

A **fresh** `newton_diff` object is created for each experiment. The derivatives - and therefore the companion matrix - depend on the values of `MUL` and `ACC`, so an object made for an earlier experiment would answer with the eigenvalues of that experiment. `<modelclass>.get_eigenvalues()` keeps its `newton_diff` object in `<modelclass>.stability_newton` and reuses it, which is what you want when the same solution is analysed repeatedly, but not here.

```
def geteigen(mul, acc, years=30, per=2023, show=False):
    '''Solves the Samuelson multiplier -accelerator model and calculates the eigenvalues
    of the companion matrix in order to evaluate stability.

    per is the year for which the companion matrix and the eigenvalues are displayed.
    Returns the newton_diff object, so that it can be used for further analysis.'''

    baseline_not_solved = get_df(mul, acc, years=years)
    baseline = mma(baseline_not_solved, silent=True)          # solve the model

    # newton_diff holds the derivative machinery. A new one is created for each
    # experiment, as the derivatives depend on the values of MUL and ACC.
    newton = mma.get_newton()

    # calculates the companion matrix and its eigenvalues and eigenvectors for each period
    _ = newton.get_eigenvalues(silent=True)

    # a dict: key is the year, value is a dataframe with eigenvalues in the first row
    # and the corresponding eigenvectors below
    eig_dict = newton.get_df_eigen_dict()

    display(Markdown(f'**Model where multiplier = {mul} and accelerator = {acc}**'))
    display(newton.eigenvector_plot(size=(4, 4), per=per))    # show the eigenvalues

    mma['gdp consumption investment'].plot(top=0.8, title='Values', colrow=3,
    share=False)      # show the solution

    if show:
        # The model is linear and time independent, so one year tells the whole story
        eigper = eig_dict[per]

        # a function which calculates a list of absolute values from a column of
        # complex numbers
        compabs = lambda complex: [abs(value) for index, value in complex.items()]

        # only retain the columns where the length of the eigenvalue is above 0.0001
```

```

eigperout = (eigper.
    T. # transpose, as we →
    query and eval on columns
    eval('absolute_value=@compabs(Eigenvalues)'). # the absolute →
    value of the eigenvalue
    query('absolute_value>=0.0001'). # keep the rows →
    where the abs value is above 0.0001
    drop('absolute_value', axis=1). # the absolute →
    value is not needed anymore
    T) # transpose back

#display(Markdown(f'**Companion matrix M in {per}**'))
#display(newton.get_df_comp_dict()[per])

display(Markdown(f'**Non zero eigenvalues and -vectors in {per}**'))
display(eigperout)

return newton

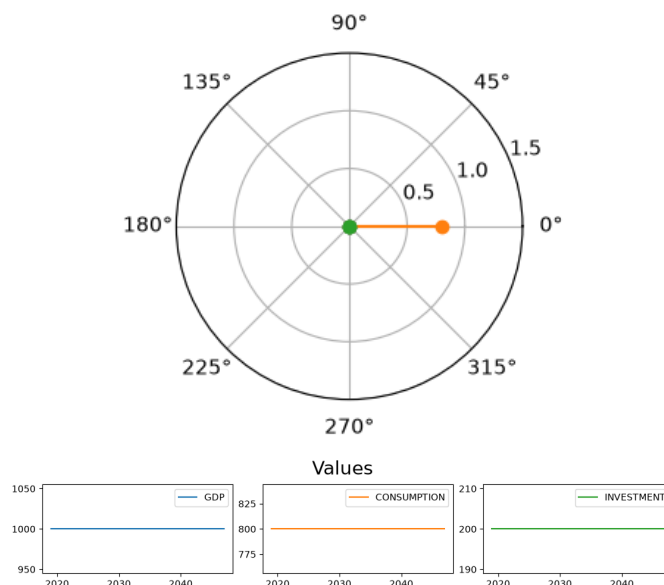
```

6.5 No amplification

```
geteigen(mul=0.8, acc=0, show=1);
```

Model where multiplier = 0.8 and accelerator = 0

Eigen values in 2023



Non zero eigenvalues and -vectors in 2023

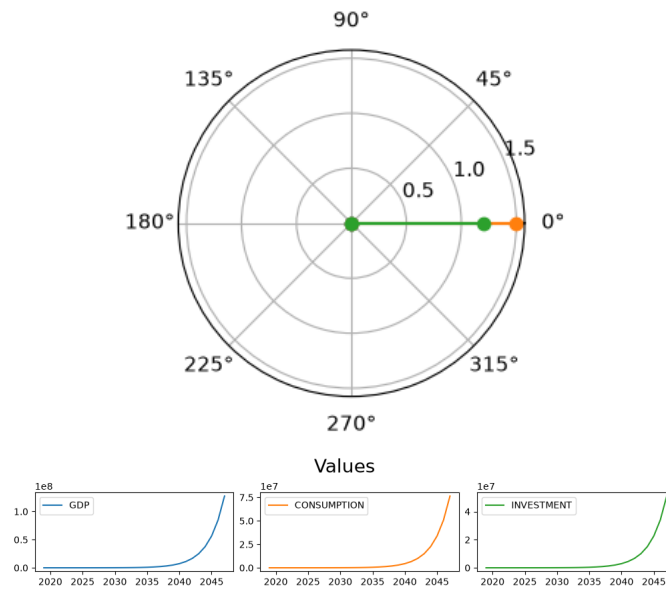
	1
Eigenvalues	0.800000
CONSUMPTION(-1)	0.707107
GDP(-1)	0.707107
INVESTMENT(-1)	0.000000

6.6 Explosion

```
geteigen(mul=0.9, acc=2, show=1);
```

Model where multiplier = 0.9 and accelerator = 2

Eigen values in 2023



Non zero eigenvalues and -vectors in 2023

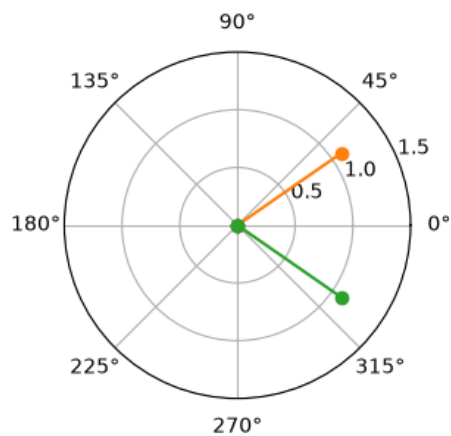
	1	2
Eigenvalues	1.500000	1.200000
CONSUMPTION(-1)	0.486664	0.588348
GDP(-1)	0.811107	0.784465
INVESTMENT(-1)	0.324443	0.196116

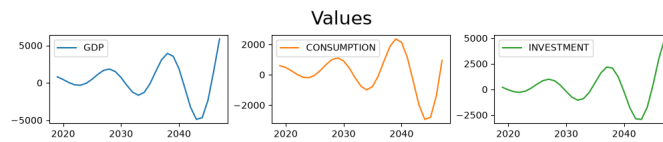
6.7 Exploding oscillations

```
| geteigen(mul=0.6, acc=2, show=1);
```

Model where multiplier = 0.6 and accelerator = 2

Eigen values in 2023





Non zero eigenvalues and -vectors in 2023

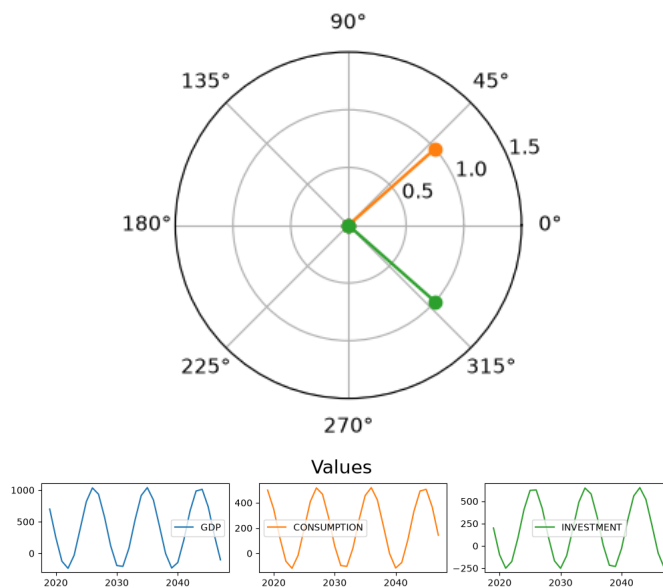
	1	2
Eigenvalues	$0.900000+0.624500j$	$0.900000-0.624500j$
CONSUMPTION(-1)	$-0.345134+0.239485j$	$-0.345134-0.239485j$
GDP(-1)	$-0.766965+0.000000j$	$-0.766965-0.000000j$
INVESTMENT(-1)	$-0.421831-0.239485j$	$-0.421831+0.239485j$

6.8 Perpetual oscillations

```
| geteigen(mul=0.5, acc=2, show=1);
```

Model where multiplier = 0.5 and accelerator = 2

Eigen values in 2023



Non zero eigenvalues and -vectors in 2023

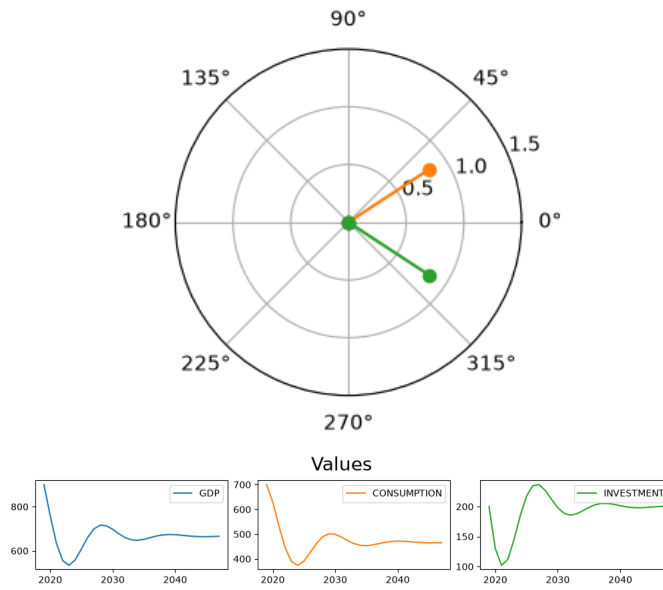
	1	2
Eigenvalues	$0.750000+0.661438j$	$0.750000-0.661438j$
CONSUMPTION(-1)	$-0.283473+0.250000j$	$-0.283473-0.250000j$
GDP(-1)	$-0.755929+0.000000j$	$-0.755929-0.000000j$
INVESTMENT(-1)	$-0.472456-0.250000j$	$-0.472456+0.250000j$

6.9 Dampened oscillations

```
| geteigen(mul=0.7, acc=1, show=1);
```

Model where multiplier = 0.7 and accelerator = 1

Eigen values in 2023



Non zero eigenvalues and -vectors in 2023

	1	2
Eigenvalues	0.700000+0.458258j	0.700000-0.458258j
CONSUMPTION(-1)	0.494975-0.324037j	0.494975+0.324037j
GDP(-1)	0.707107+0.000000j	0.707107-0.000000j
INVESTMENT(-1)	0.212132+0.324037j	0.212132-0.324037j

6.10 Jackknife experiments to identify the impact from each equation

To understand the impact of each equation a jackknife can be made. This is done by deleting the row and the column belonging to each equation in turn and recording the new eigenvalues.

The idea is to identify the equations which contribute most to the eigenvalues of the whole system.

The `newton_diff` object returned above also performs this operation. `newton.get_eigen_jackknife_abs_select(year)` returns the sum of the **largest** absolute eigenvalues when each equation in turn is deleted. The row named `NONE` is the value when no equation is deleted.

Note

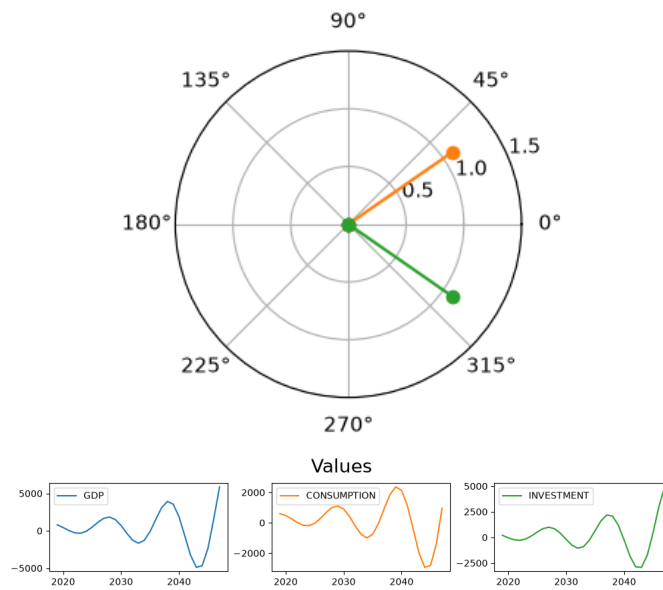
The terminology is loosely coined from Jackknife resampling Jackknife resampling.

Contributions to better naming are very welcome.

```
newton = geteigen(mul=0.6, acc=2, show=1)
newton.get_eigen_jackknife_abs_select(year=2023, largest=2)
```

Model where multiplier = 0.6 and accelerator = 2

Eigen values in 2023



Non zero eigenvalues and -vectors in 2023

	1	2
Eigenvalues	0.900000+0.624500j	0.900000-0.624500j
CONSUMPTION(-1)	-0.345134+0.239485j	-0.345134-0.239485j
GDP(-1)	-0.766965+0.000000j	-0.766965-0.000000j
INVESTMENT(-1)	-0.421831-0.239485j	-0.421831+0.239485j

Calculating eigenvalues of 3 different matrices takes time, so make cup of coffee and →
a take a short nap

0% | 0/3 [00:00<?, ?it/s]

```
CONSUMPTION    0.00000
GDP             0.00000
INVESTMENT     0.60000
NONE           2.19089
dtype: float64
```

The output above shows that the largest eigenvalue of the system is 1.1, so the system is unstable. If either the consumption or the GDP equation is deleted, the largest $|\lambda|$ will be 0, so the system will not be unstable. The same goes if the investment equation is excluded - however in that case the largest $|\lambda|$ will be 0.6.

So for such a small model the procedure is not very useful. But for larger models it can help to identify rough equations.

Bibliography

- S. Adjemian, H. Bastani, M. Juillard, F. Karamé, J. Maih, F. Mihoubi, W. Mutschler, G. Perendia, J. Pfeifer, M. Ratto, and S. Villemot. Dynare: Reference Manual Version 4. Dynare Working Paper 1, CEPREMAP, 2011. URL <https://www.dynare.org/wp-repo/dynarewp001.pdf>. Revised March 2021.
- F. J. H. Don. Some issues in solving large sparse systems of equations. *Journal of Economic Dynamics and Control*, 14:313–325, 1990. ISSN 0165-1889.
- A. Drud. A survey of model representations and simulation algorithms in some existing modeling systems. *Journal of Economic Dynamics and Control*, 5:5–35, 1983. ISSN 0165-1889. Surveys sixteen modeling systems, restricted to systems based on time series models.
- T. Frentz. Tekstbehandlingssystemet MACRO. RECKU publikation 86, RECKU, 1981. 2. udgave, kategori SK/D.
- I. Hansen. Brugerhåndbog Del II: Simulationsprogrammet PCIM. User handbook, Danmarks Statistik, Modelgruppen, 9 1999. URL <https://www.dst.dk/Site/Dst/SingleFiles/GetArchiveFile.aspx?fi=297931410>. Published without attribution.
- T. M. Kristensen. Introduktion til ADAM kørsler i PCIM. Modelgruppepapir, Danmarks Statistik, Modelgruppen, 1 1996. URL <https://www.dst.dk/Site/Dst/SingleFiles/GetArchiveFile.aspx?fi=adam&fo=tmk31196--pdf>.
- G. Pauletto. *Computational Solution of Large-Scale Macroeconometric Models*. Kluwer Academic Publishers, 1 1997. ISBN 978-1-4419-4778-9. doi: 10.1007/978-1-4757-2631-2_3.
- C. E. Petersen and C. A. Sims. Computer simulation of large-scale econometric models: Project link. *International Journal of High Performance Computing Applications*, 1(4), 1987. ISSN 17412846. doi: 10.1177/109434208700100404. URL (<http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.853.6387&rep=rep1&type=pdf>).
- C. G. Renfro. A compendium of existing econometric software packages. *Journal of Economic and Social Measurement*, 29:359–409, 2004. ISSN 0747-9662. Supplement to the survey Econometric software: the first fifty years in perspective.